

Domain Name System

25.1 Name Space

Flat Name Space

Hierarchical Name Space

25.2 Domain Name Space

Label

Domain Name

Figure 25.1 Domain name space

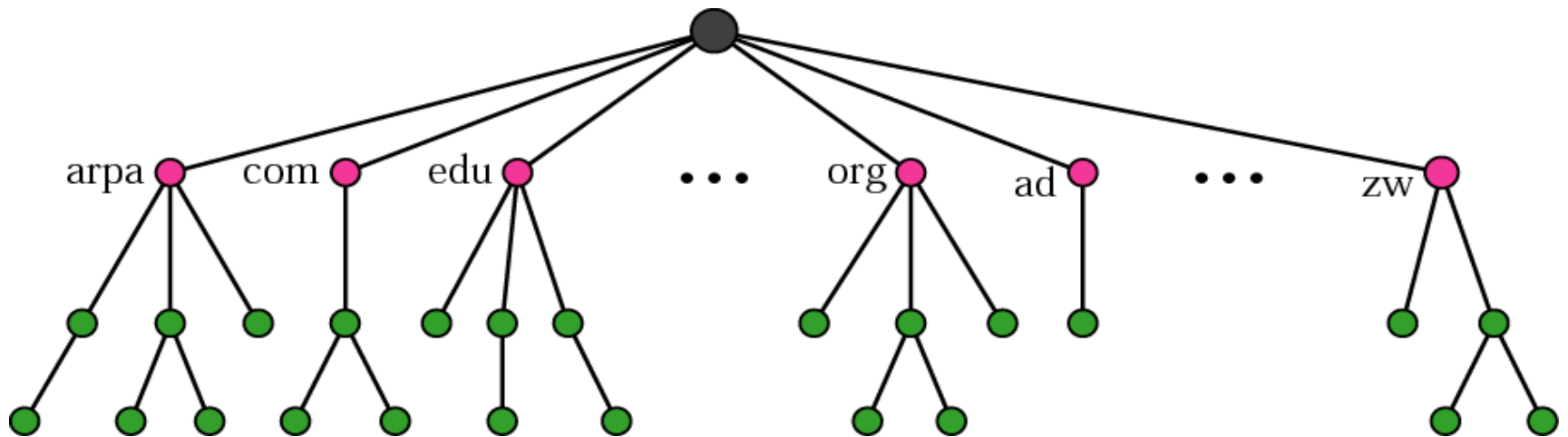
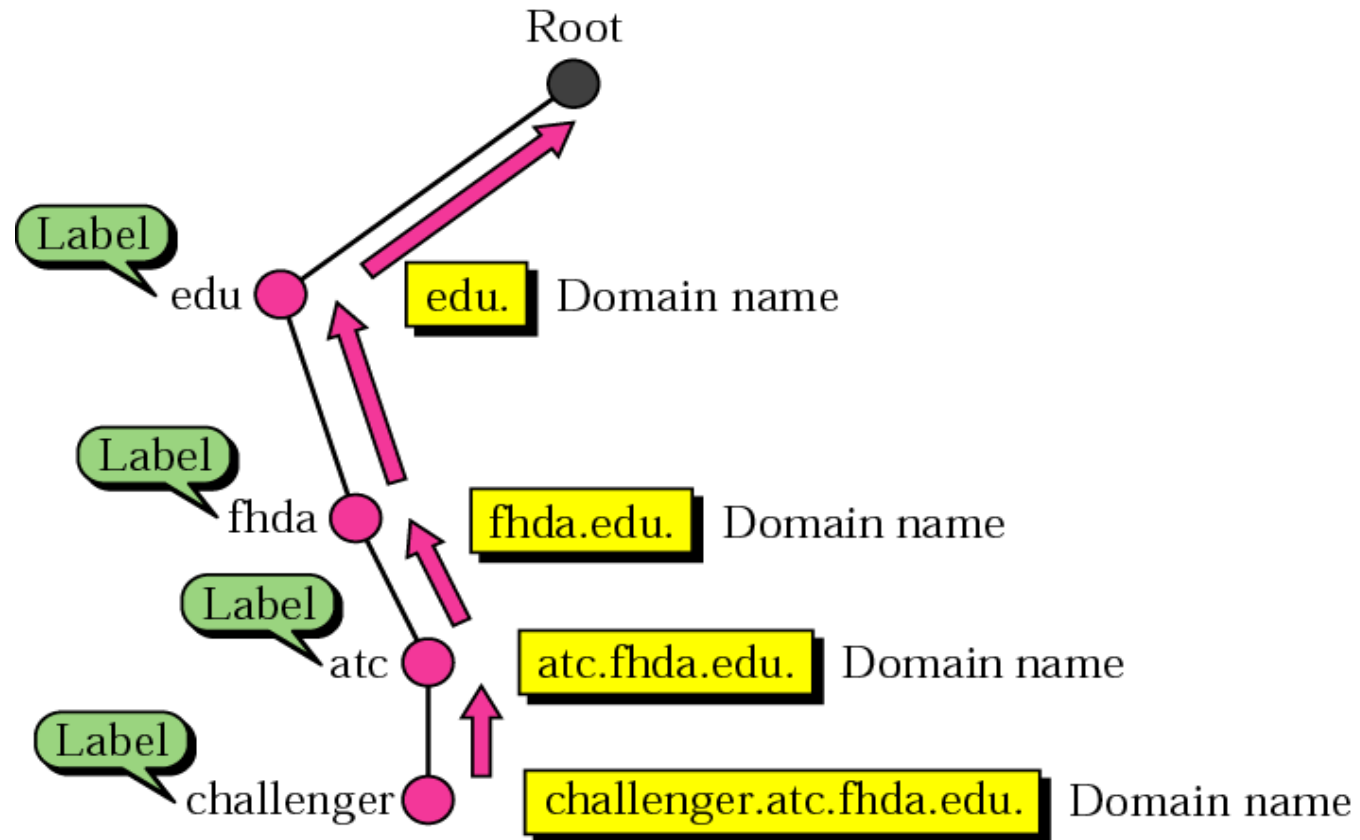


Figure 25.2 Domain names and labels



25.6 DNS Messages

Header

Question Section

Answer Section

Authoritative Section

Additional Information Section

Figure 25.3 FQDN and PQDN

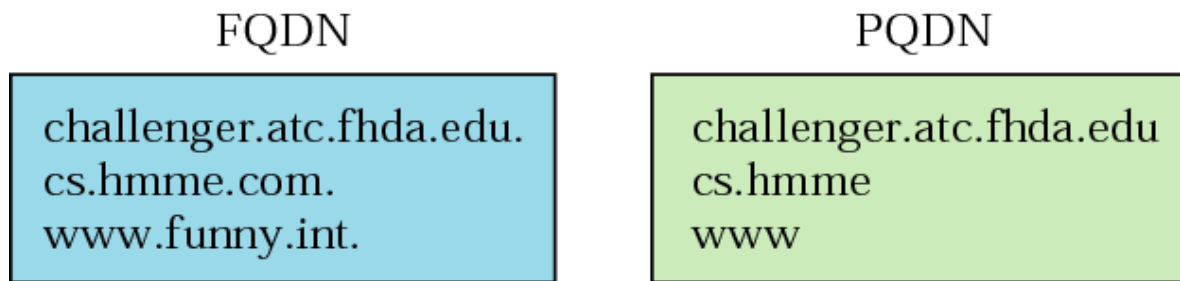
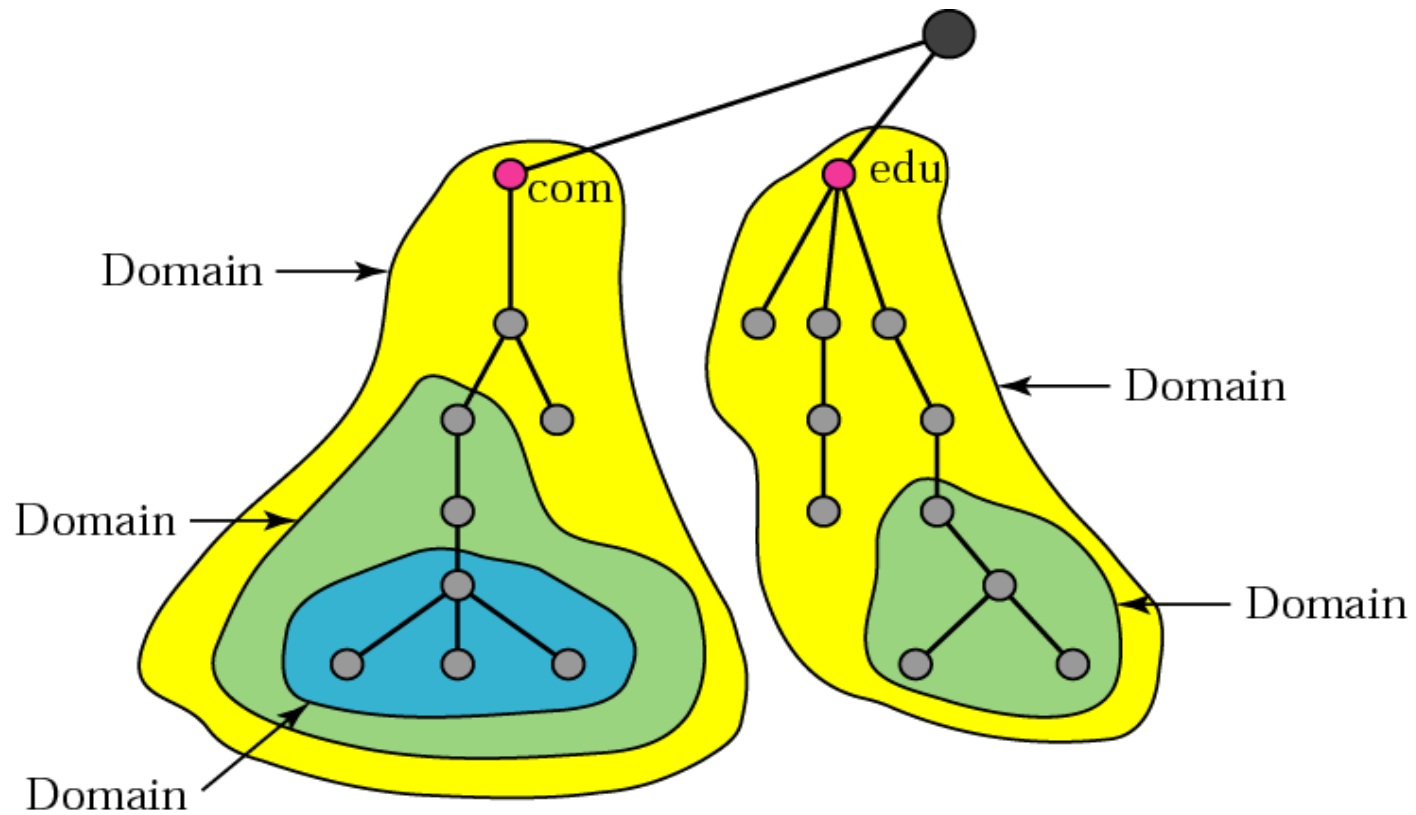


Figure 25.4 Domains



25.3 Distribution of Name Spaces

Hierarchy of Name Servers

Zone

Root Server

Primary and Secondary Servers

Figure 25.5 Hierarchy of name servers

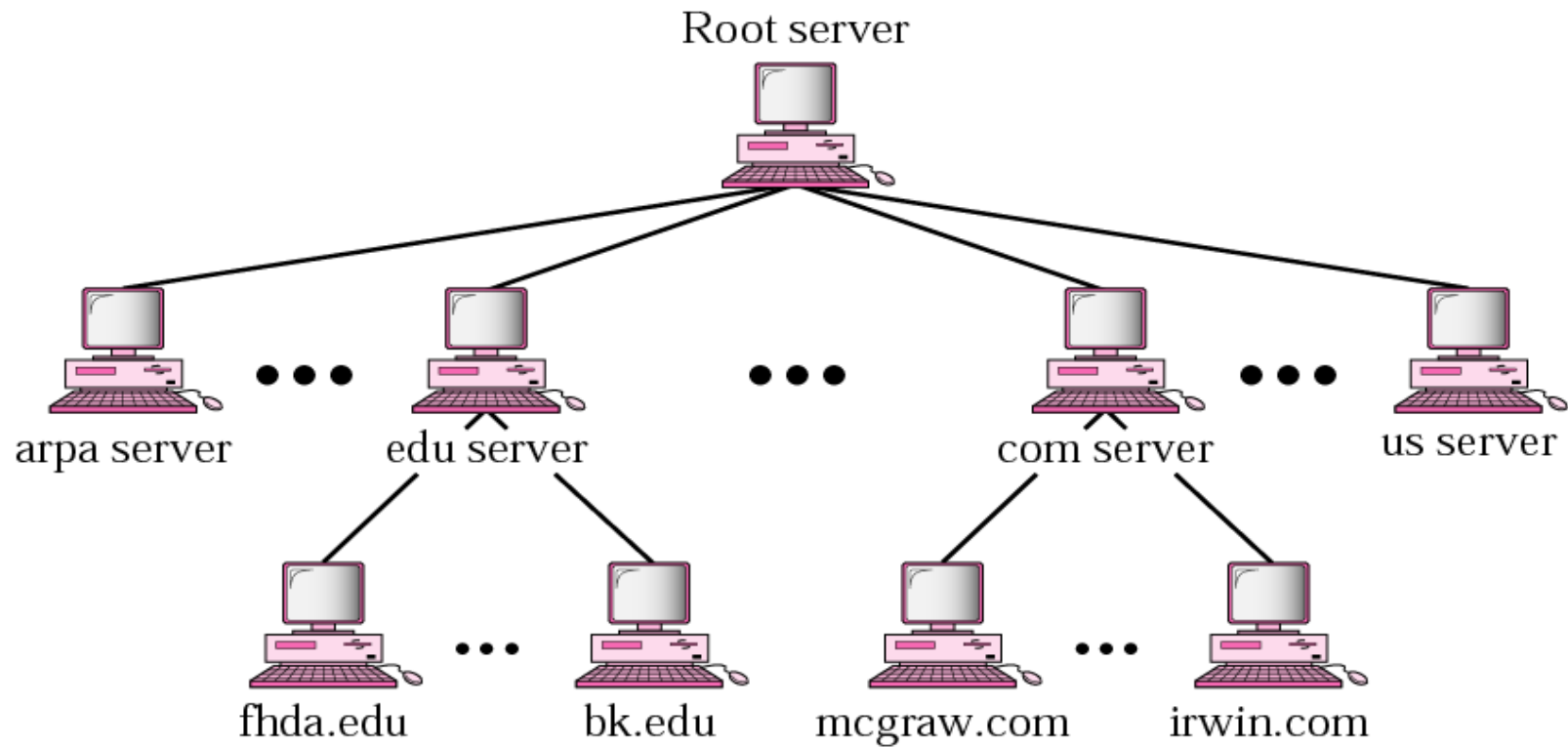
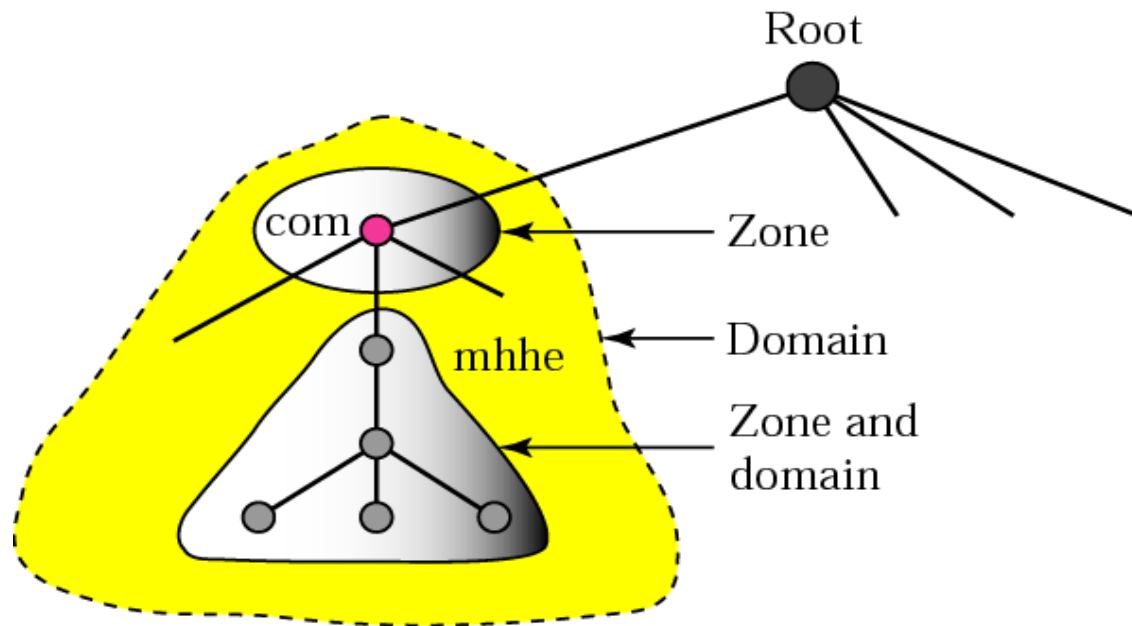


Figure 25.6 Zones and domains





A primary server loads all information from the disk file; the secondary server loads all information from the primary server.

25.4 DNS In The Internet

Generic Domain

Country Domain

Inverse Domain

Figure 25.7 DNS in the Internet

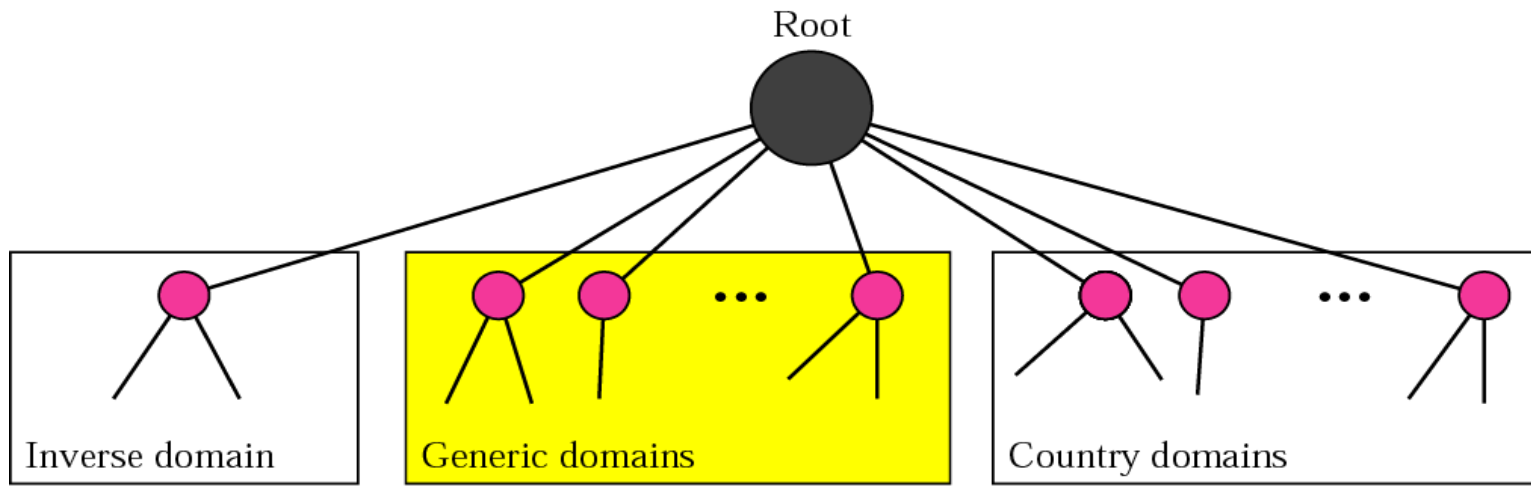


Figure 25.8 Generic domains

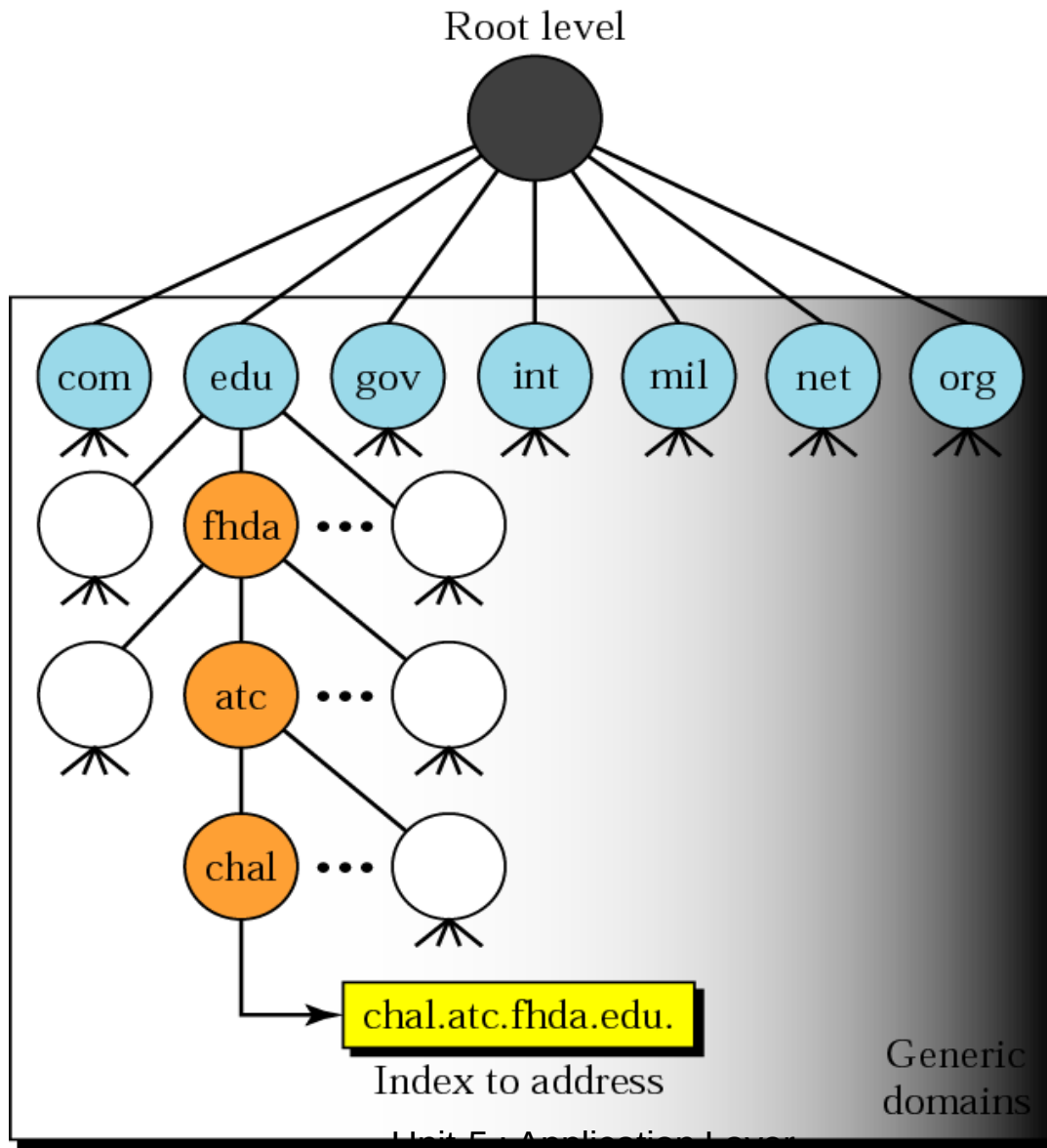


Table 25.1 *Generic domain labels*

Label	Description
com	Commercial organizations
edu	Educational institutions
gov	Government institutions
int	International organizations
mil	Military groups
net	Network support centers
org	Nonprofit organizations

Table 25.2 *New generic domain labels*

Label	Description
aero	Airlines and aerospace companies
biz	Businesses or firms (similar to com)
coop	Cooperative business organizations
info	Information service providers
museum	Museums and other nonprofit organizations
name	Personal names (individuals)
pro	Professional individual organizations

Figure 25.9 Country domains

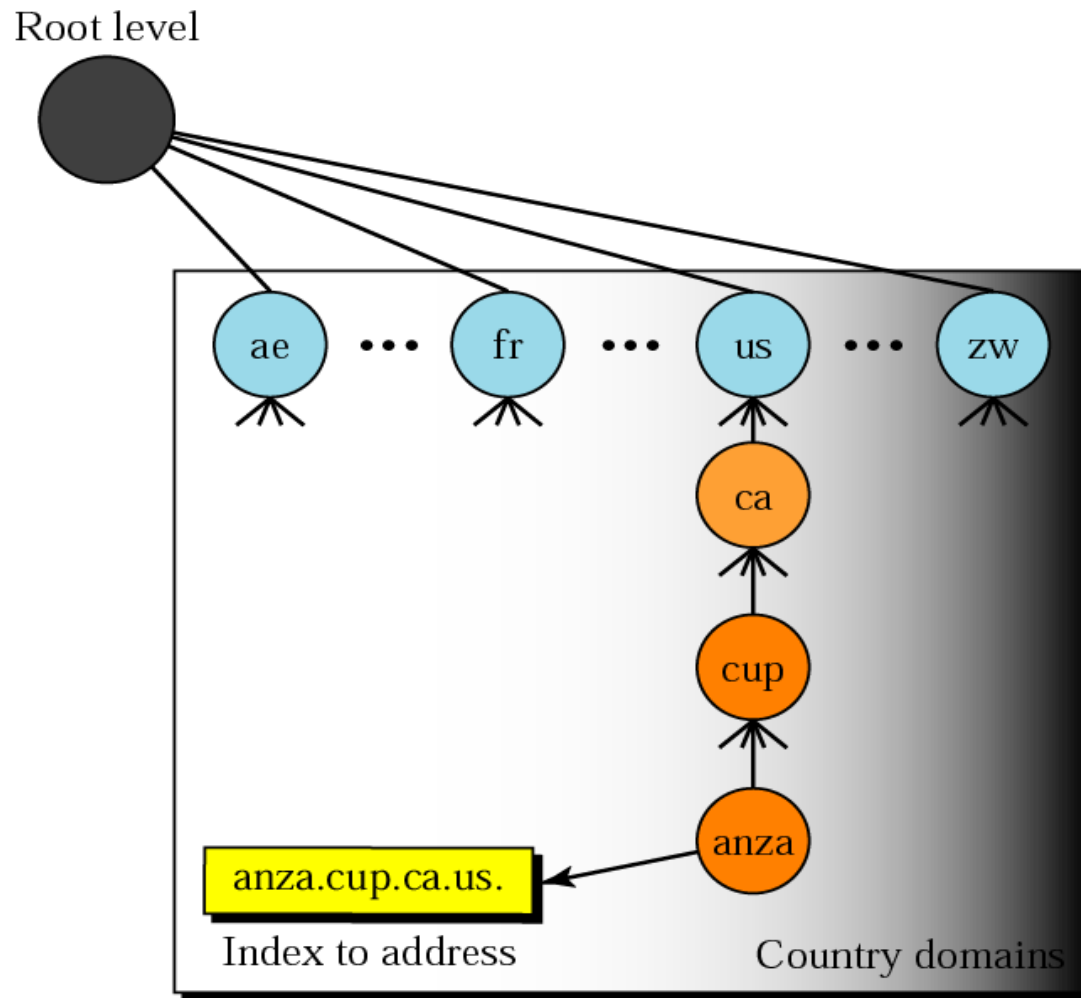
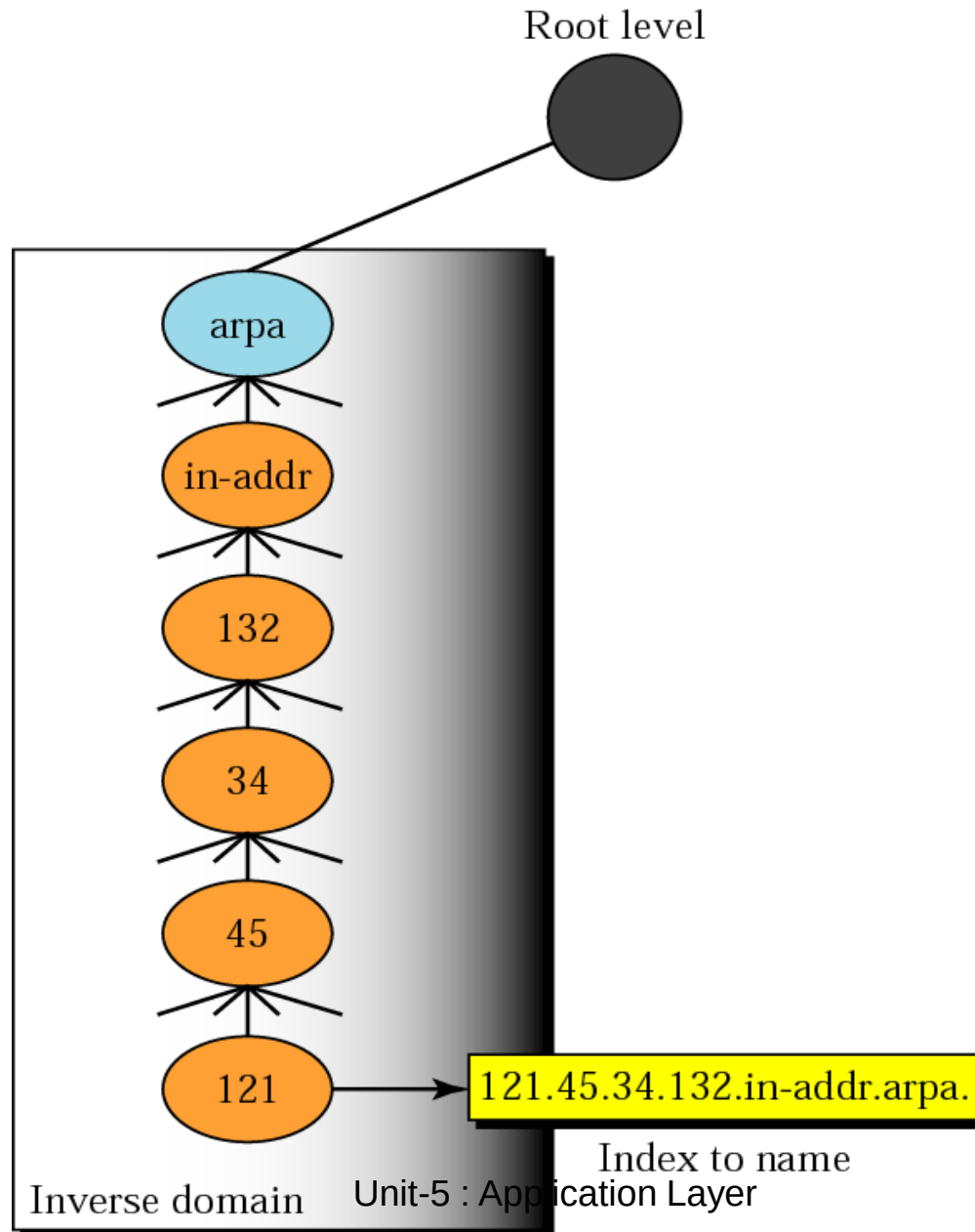


Figure 25.10 Inverse domain



25.5 Resolution

Resolver

Mapping Names to Addresses

Mapping Addresses to Names

Recursive Resolution

Iterative Resolution

Caching

Figure 25.11 Recursive resolution

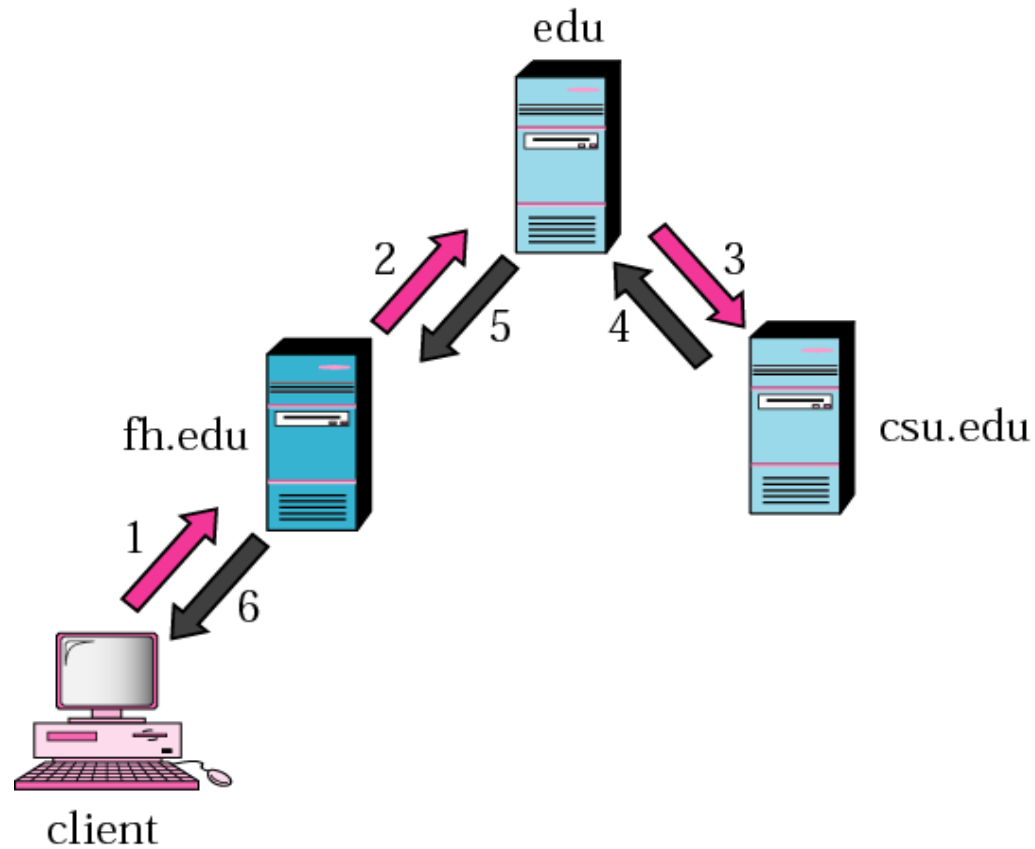


Figure 25.12 Iterative resolution

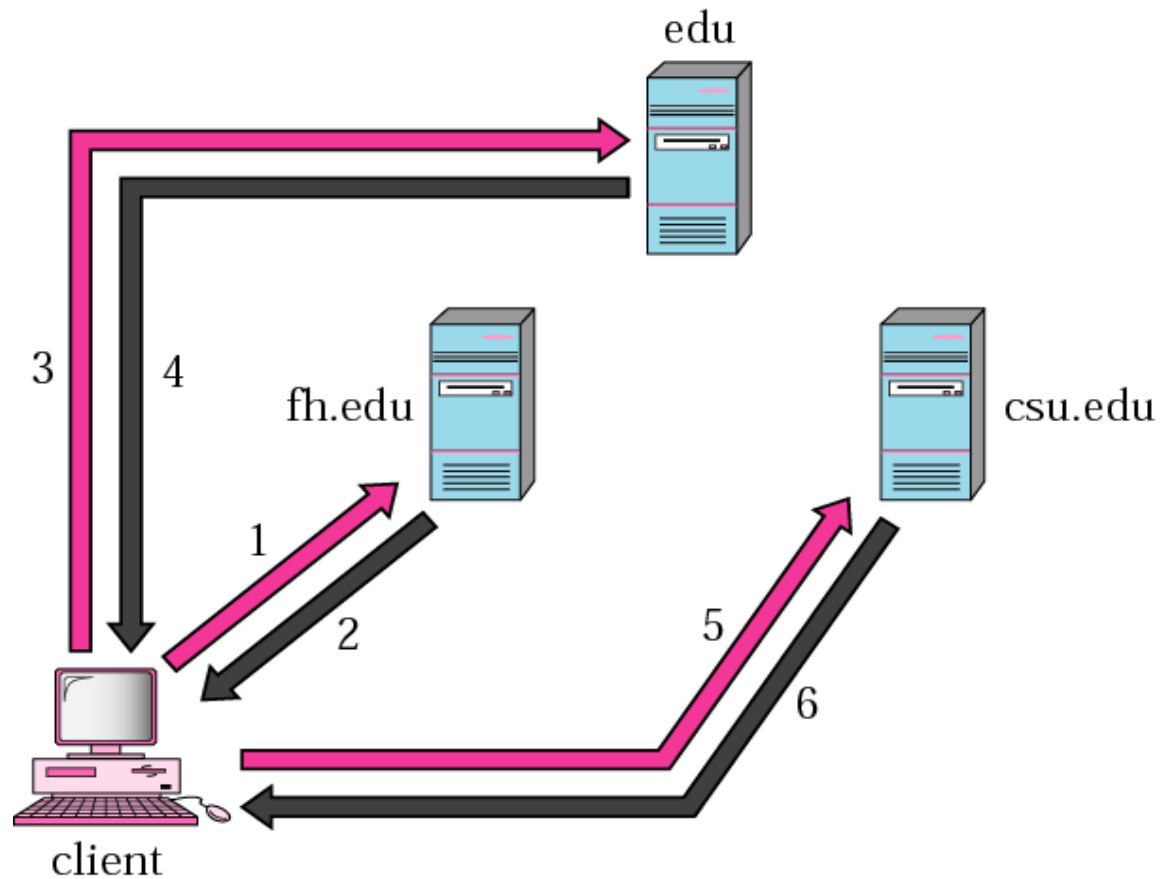
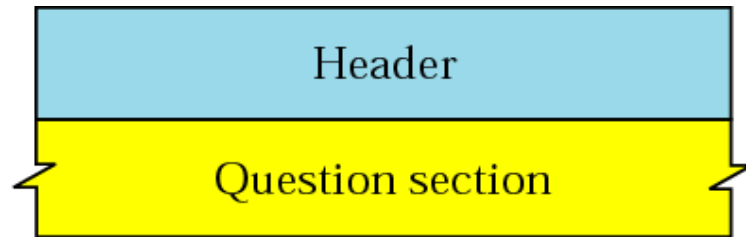
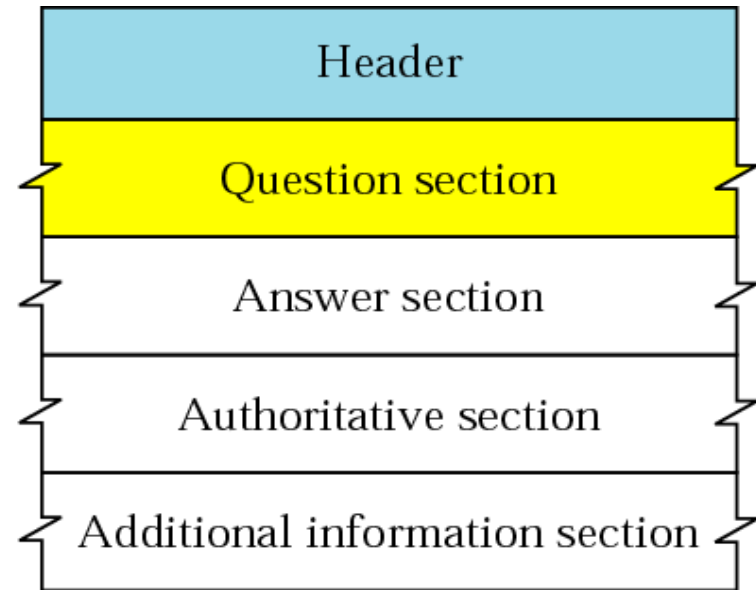


Figure 25.13 Query and response messages

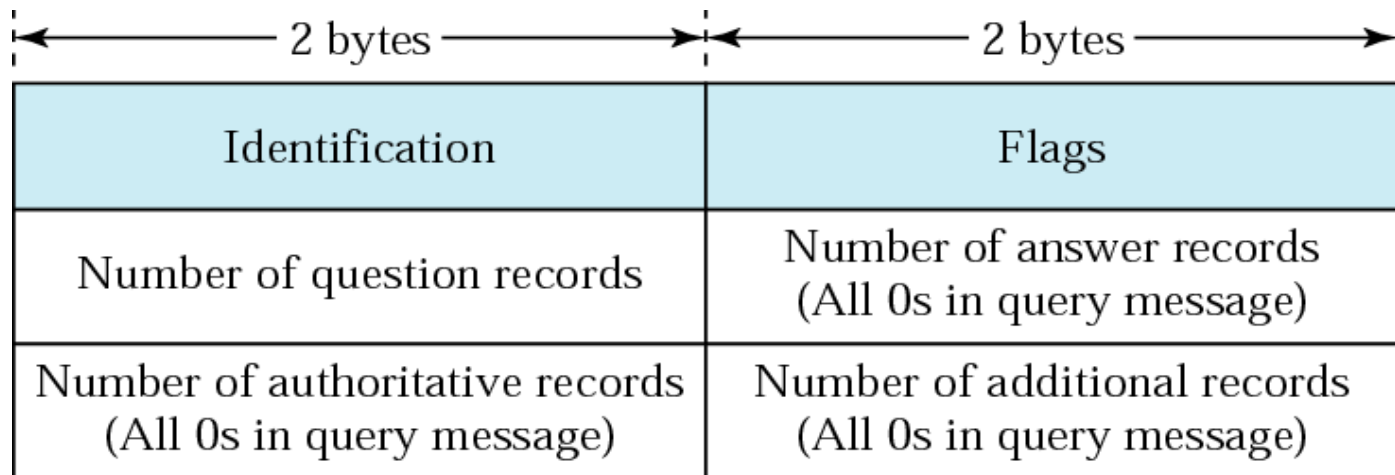


a. Query



b. Response

Figure 25.14 Header format





***DNS can use the services of
UDP or TCP,
using the well-known port 53.***

SMTP and FTP

26.1 Electronic Mail

Sending/Receiving Mail

Addresses

User Agent

MIME

Mail Transfer Agent

Mail Access Protocols

Figure 26.1 Format of an email

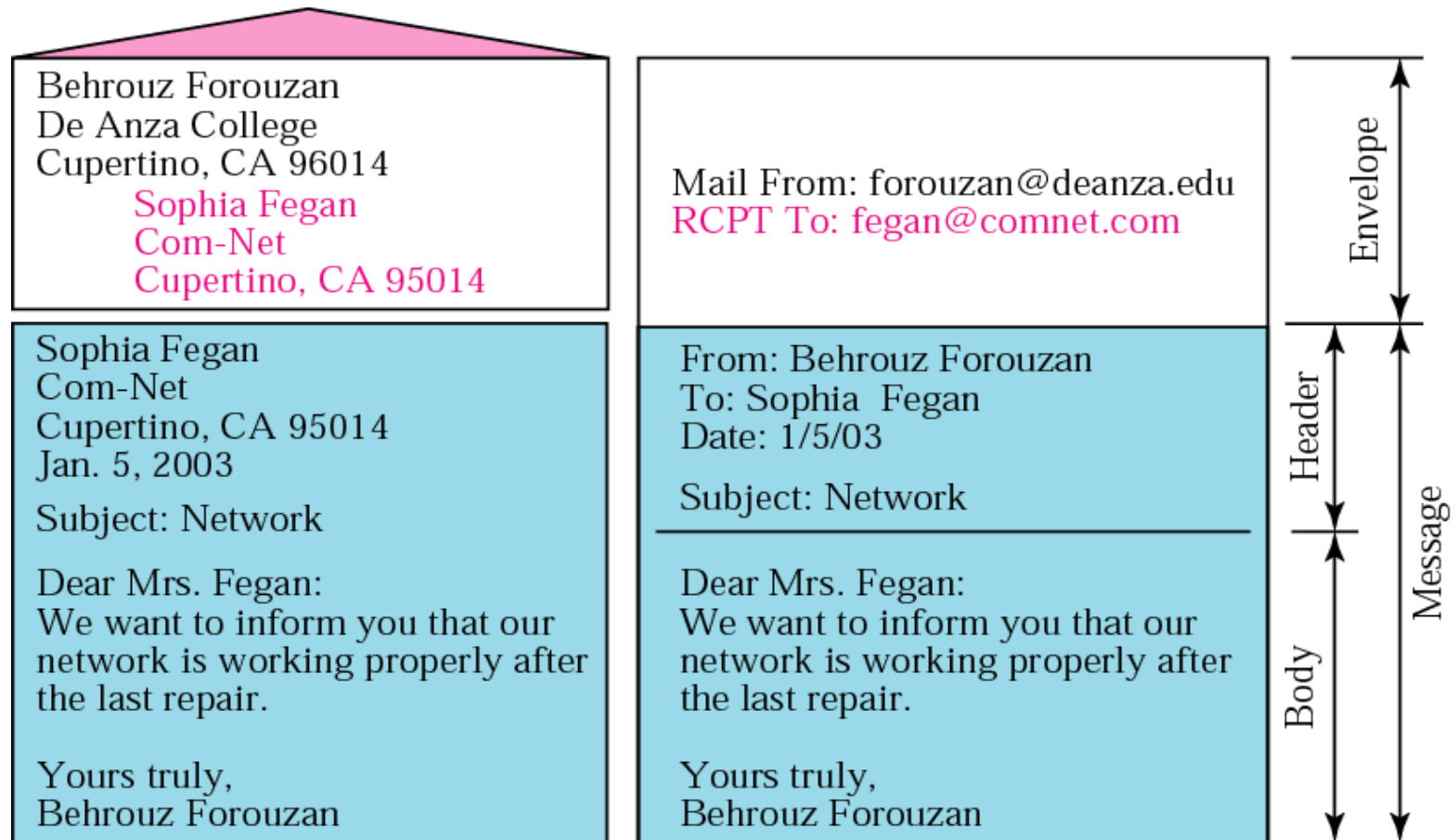


Figure 26.2 Email address

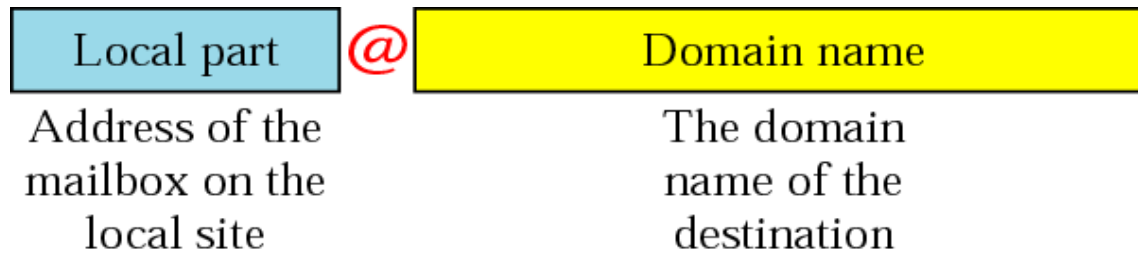
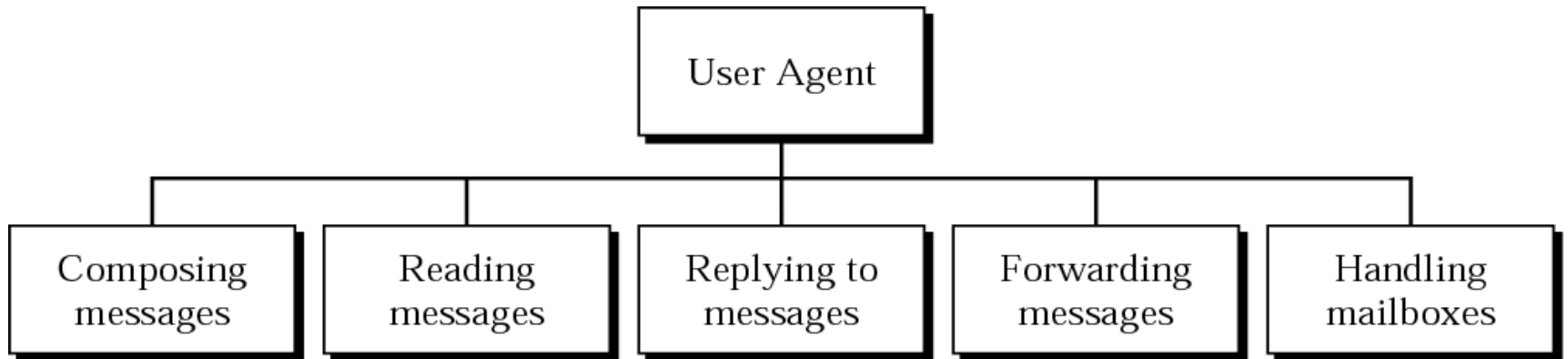


Figure 26.3 User agent





Some examples of command-driven user agents are mail, pine, and elm.



Some examples of GUI-based user agents are Eudora, Outlook, and Netscape.

Figure 26.4 MIME

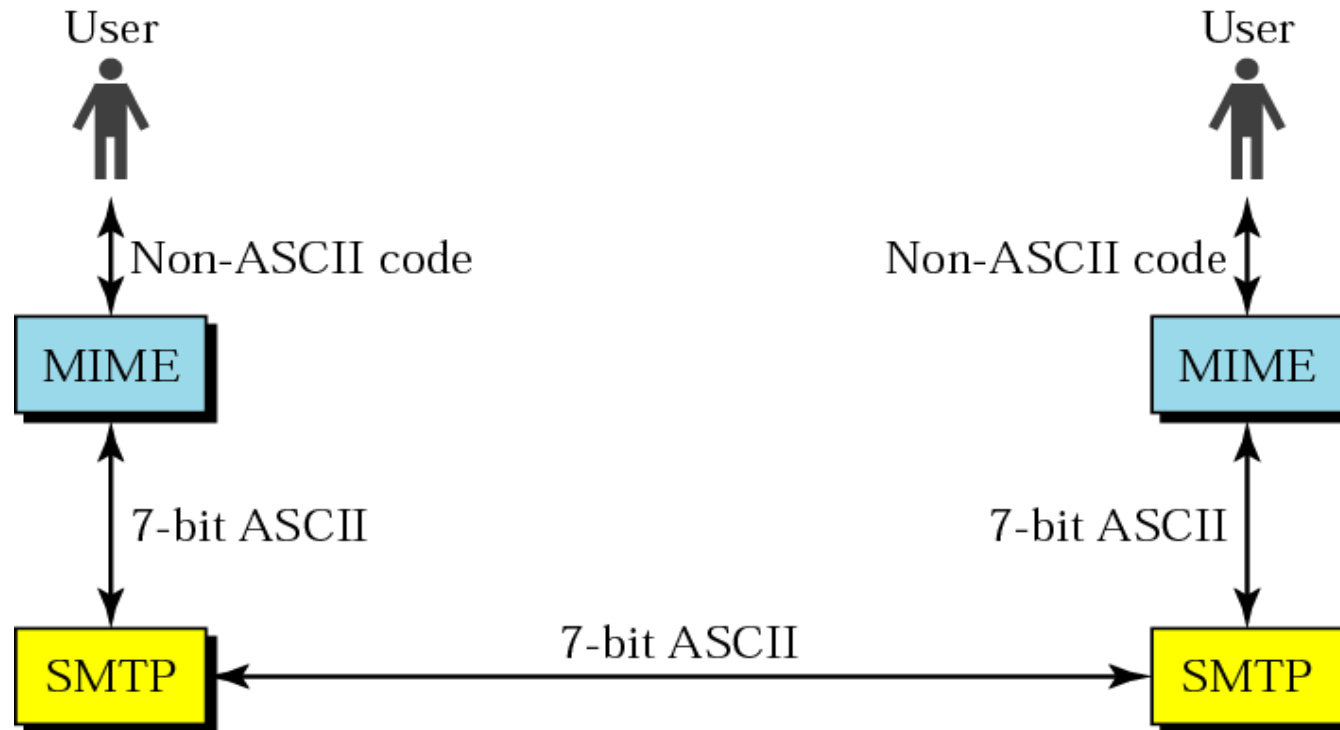
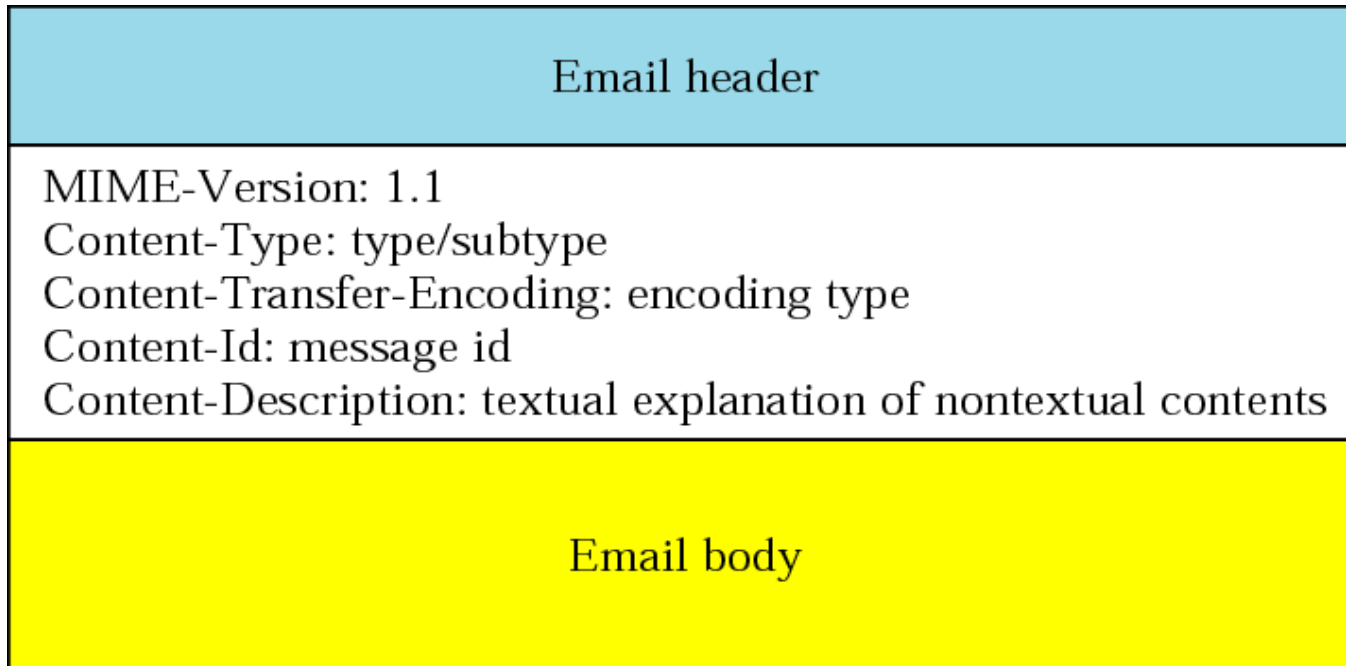


Figure 26.5 MIME header



MIME header

Table 26.1 *Data types and subtypes in MIME*

Type	Subtype	Description
Text	Plain	Unformatted text
Multiport	Mixed	Body contains ordered parts of different data types
	Parallel	Same as above, but no order
	Digest	Similar to mixed, but the default is message/RFC822
	Alternative	Parts are different versions of the same message
Message	RFC822	Body is an encapsulated message
	Partial	Body is a fragment of a bigger message
	Ext. Body	Body is a reference to another message
Image	JPEG	Image is in JPEG
	GIF	Video is in GIF format
Video	MPEG	Video is in MPEG format
Audio	Basic	Single-channel encoding of voice at 8 KHz
Application	PostScript	Adobe PostScript
	Octet-Stream	General binary data (8-bit bytes)

Table 26.2 *Content-transfer encoding*

Category	Description
Type	ASCII characters and short lines
7bit	Non-ASCII characters and short lines
8bit	Non-ASCII characters with unlimited-length lines
Binary	6-bit blocks of data are encoded into 8-bit ASCII characters
Base64	Non-ASCII characters are encoded as an equal sign followed by an ASCII code

Figure 26.6 Base64

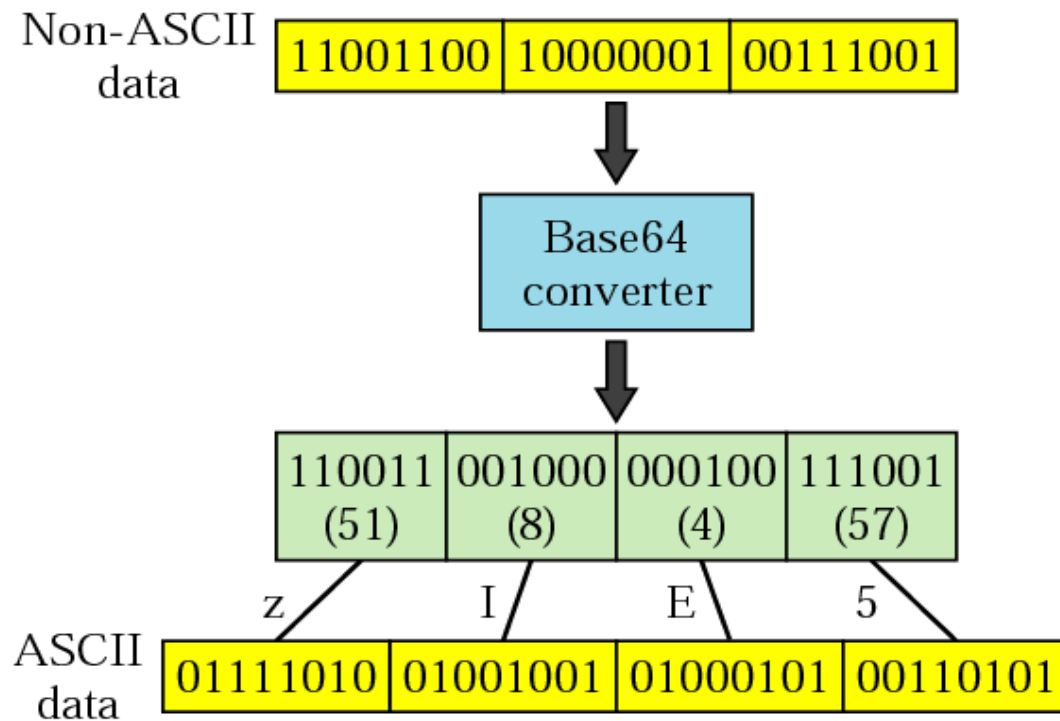


Table 26.3 *Base64 encoding table*

Value	Code	Value	Code	Value	Code	Value	Code	Value	Code	Value	Code
0	A	11	L	22	W	33	h	44	s	55	3
1	B	12	M	23	X	34	i	45	t	56	4
2	C	13	N	24	Y	35	j	46	u	57	5
3	D	14	O	25	Z	36	k	47	v	58	6
4	E	15	P	26	a	37	l	48	w	59	7
5	F	16	Q	27	b	38	m	49	x	60	8
6	G	17	R	28	c	39	n	50	y	61	9
7	H	18	S	29	d	40	o	51	z	62	+
8	I	19	T	30	e	41	p	52	0	63	/
9	J	20	U	31	f	42	q	53	1		
10	K	21	V	32	g	43	r	54	2		

Figure 26.7 Quoted-printable

Mixed ASCII and non-ASCII data

00100110 &	01001100 L	10011101 Non-ASCII	00111001 9	01001011 K
---------------	---------------	-----------------------	---------------	---------------



Quoted-printable



00100110 &	01001100 L	00111101 5	00111001 9	01000100 D	00111001 9	01001011 K
---------------	---------------	---------------	---------------	---------------	---------------	---------------

ASCII data

Figure 26.8 MTA client and server

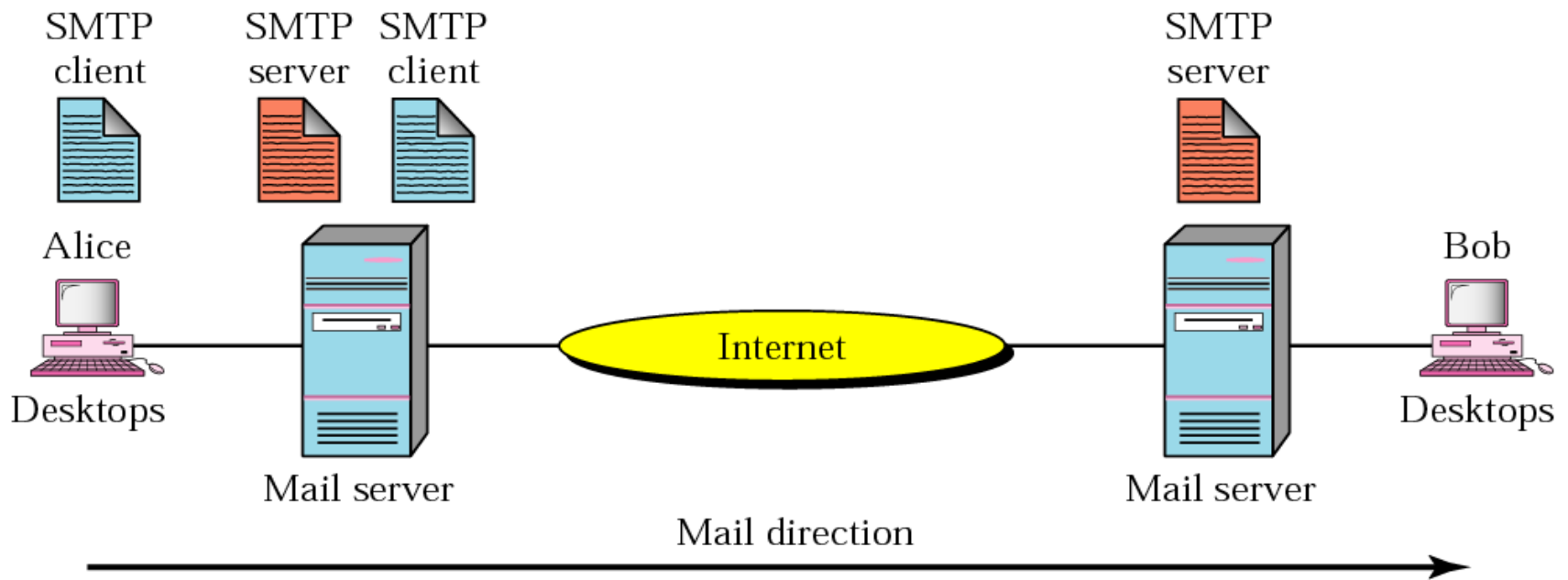


Figure 26.9 Commands and responses

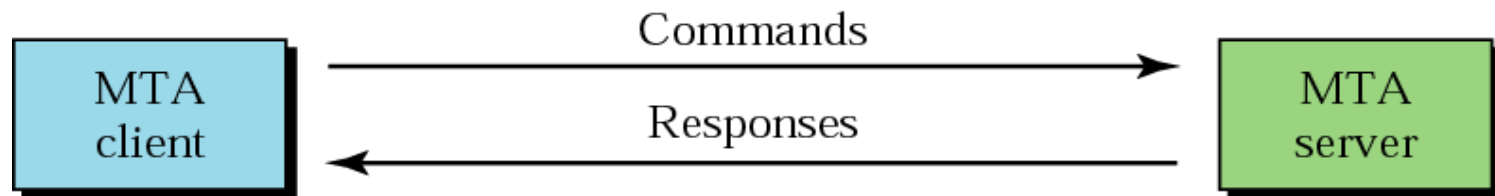


Figure 26.10 Email delivery

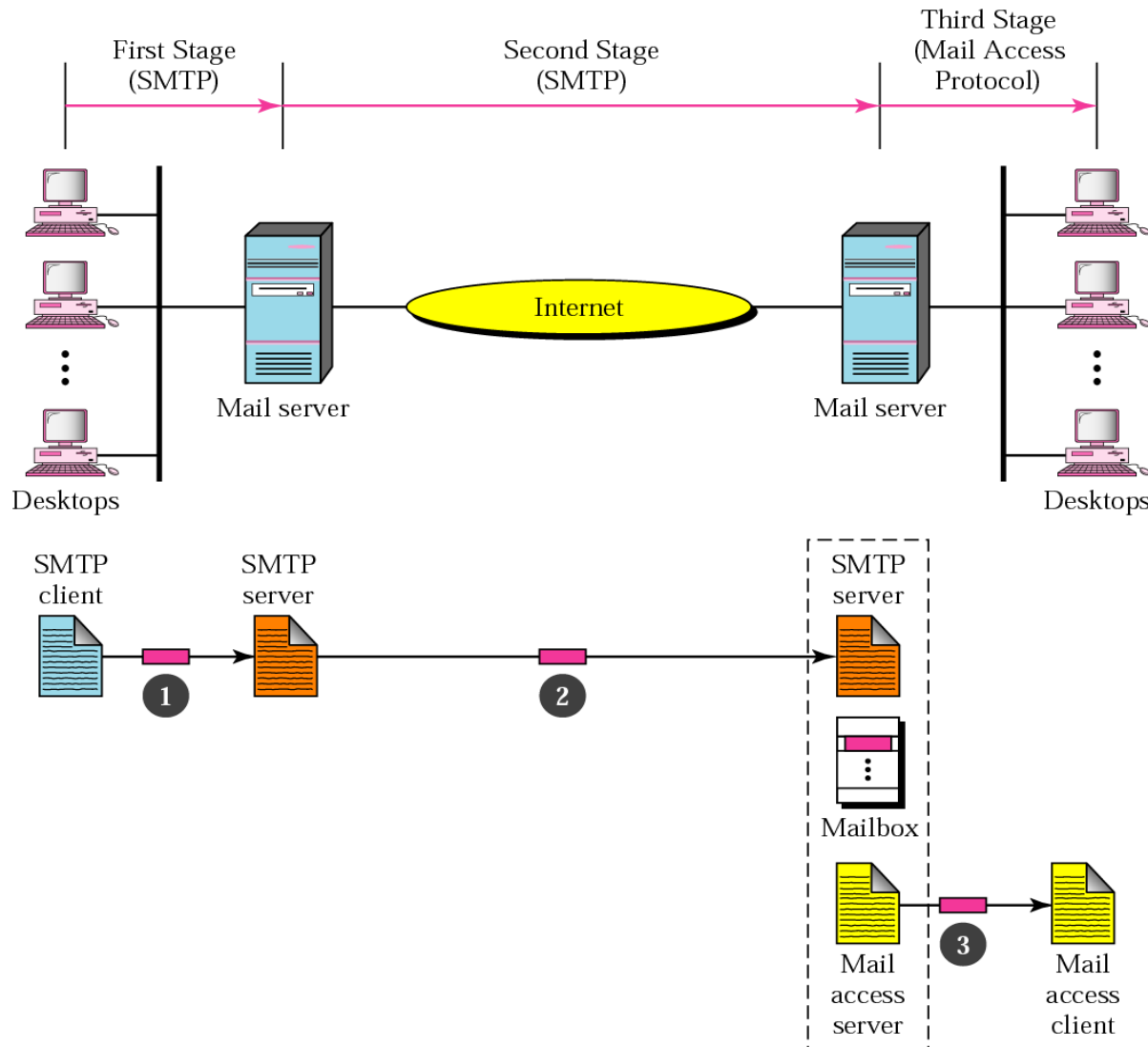
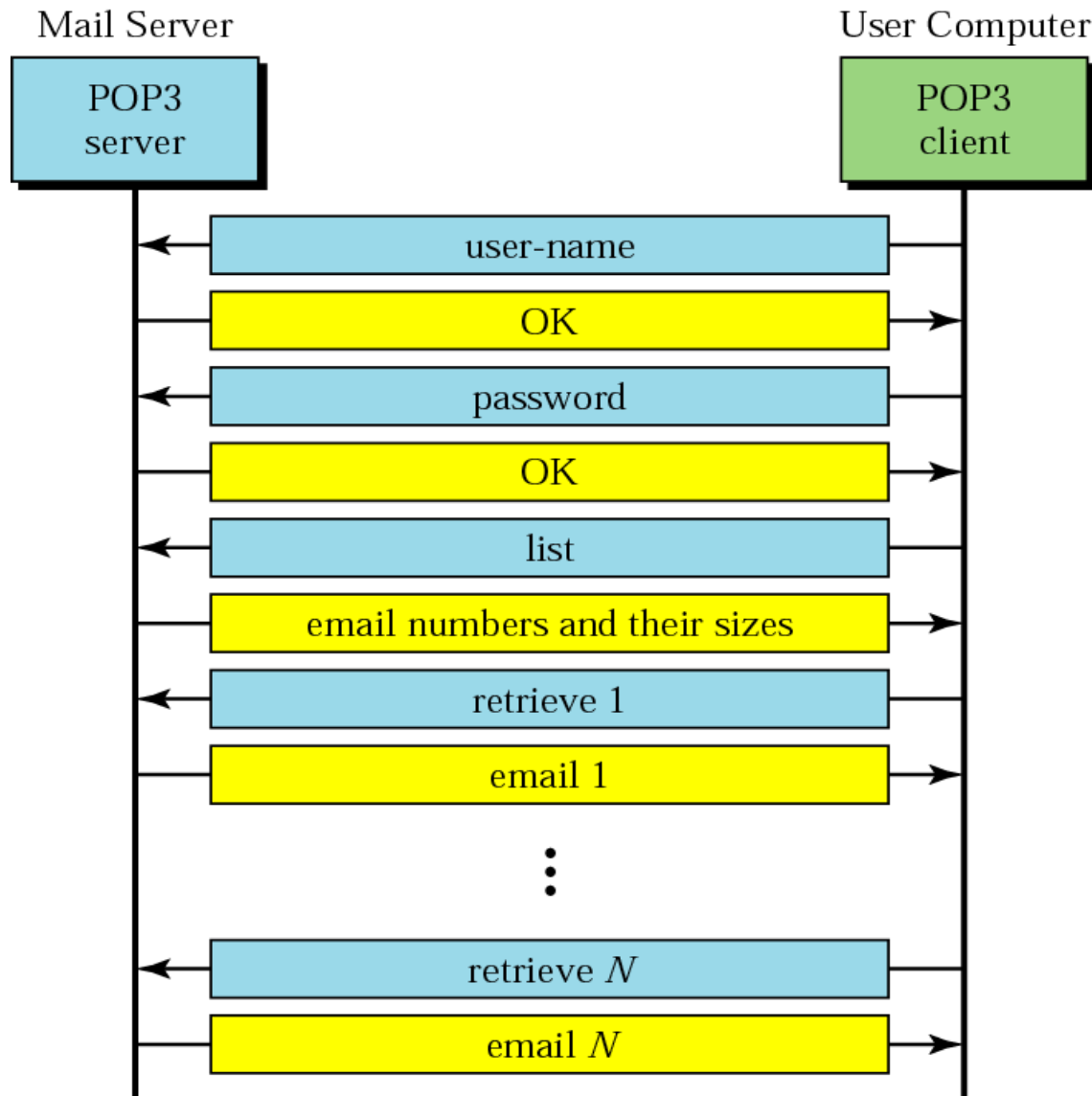


Figure 26.11 POP3



26.2 File Transfer

Connections

Communication

File Transfer

User Interface

Anonymous



FTP uses the services of TCP. It needs two TCP connections. The well-known port 21 is used for the control connection, and the well-known port 20 is used for the data connection.

Figure 26.12 FTP

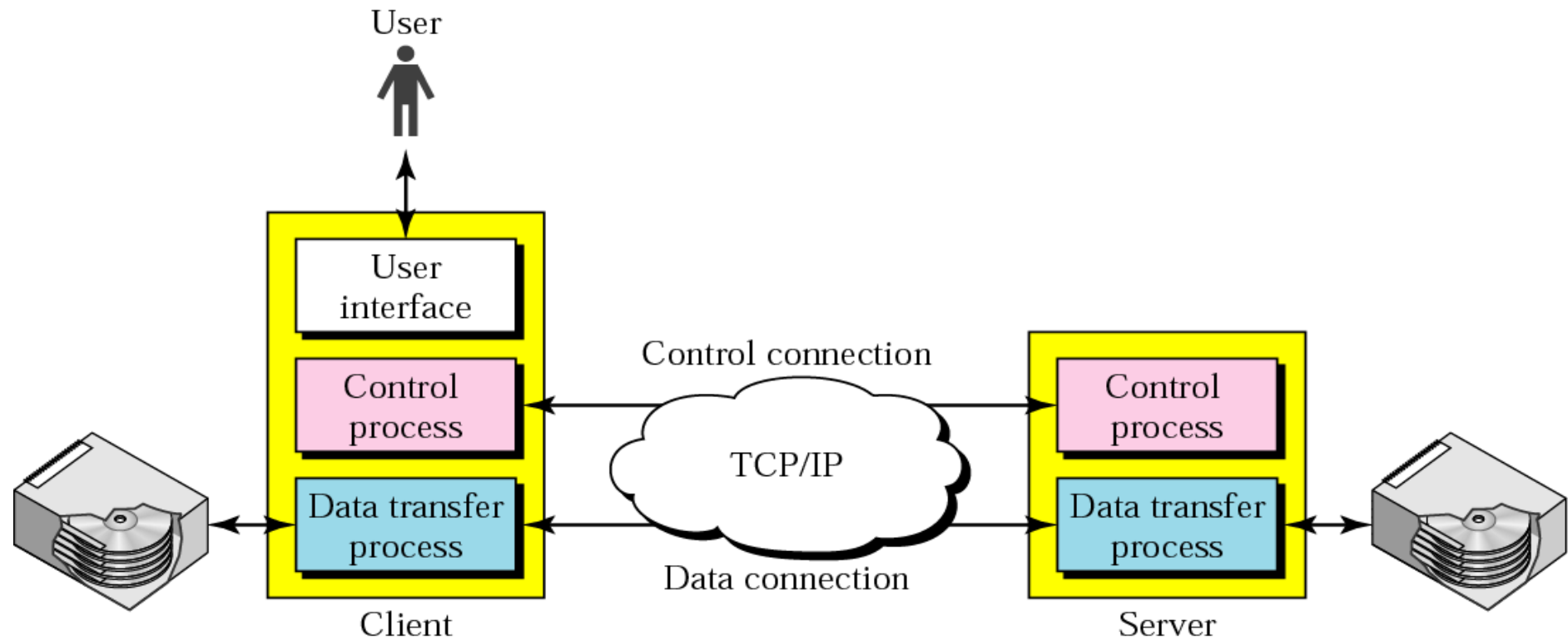


Figure 26.13 Using the control connection

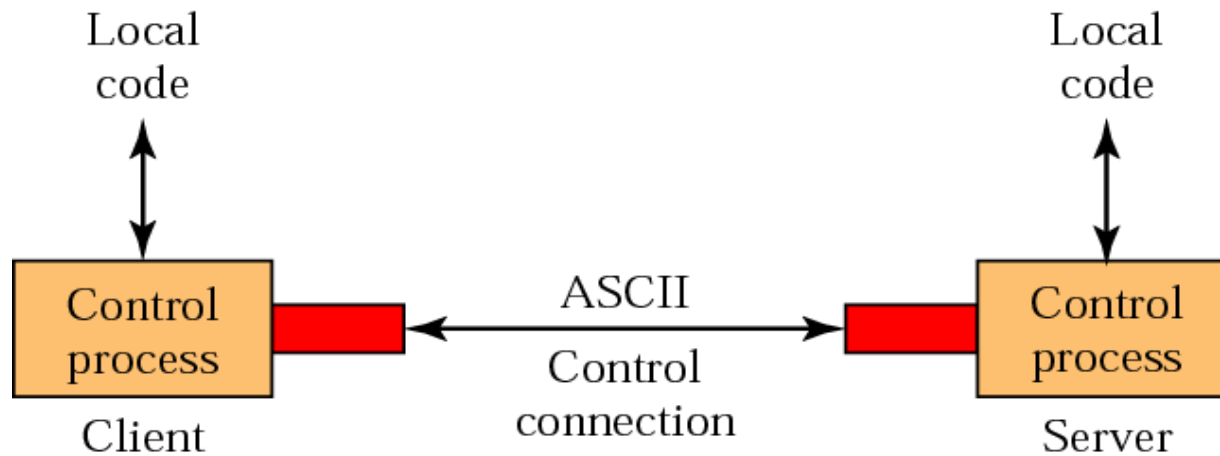


Figure 26.14 Using the data connection

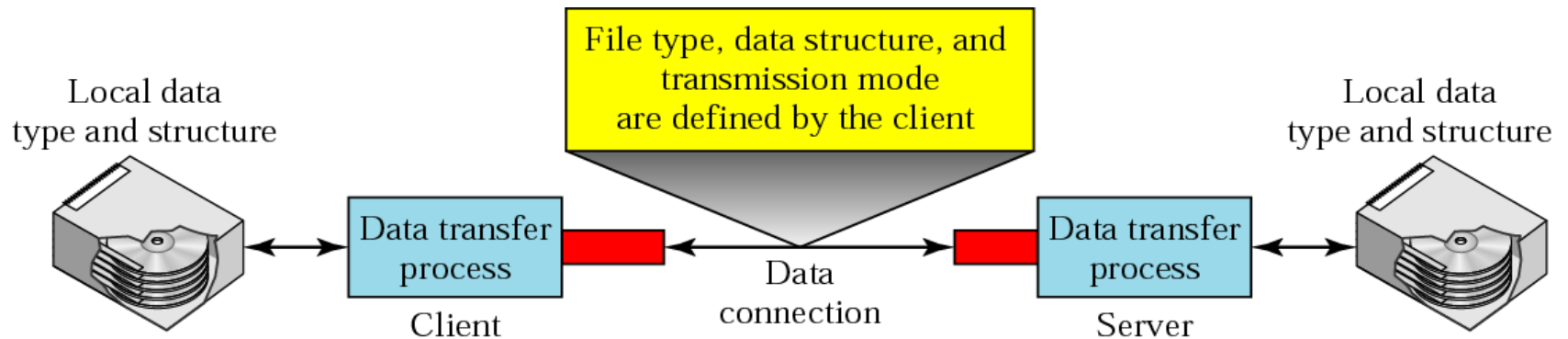
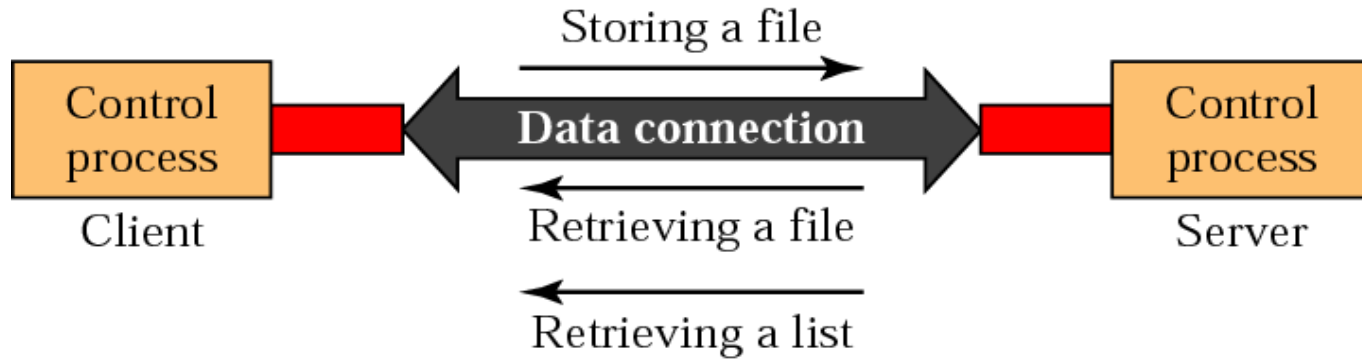


Figure 26.15 File transfer



Example 1

Figure 26.16 (next slide) shows an example of how a file is stored.

1. The control connection is created, and several control commands and responses are exchanged.
2. Data are transferred record by record.
3. A few commands and responses are exchanged to close the connection.

Figure 26.16 Example 1

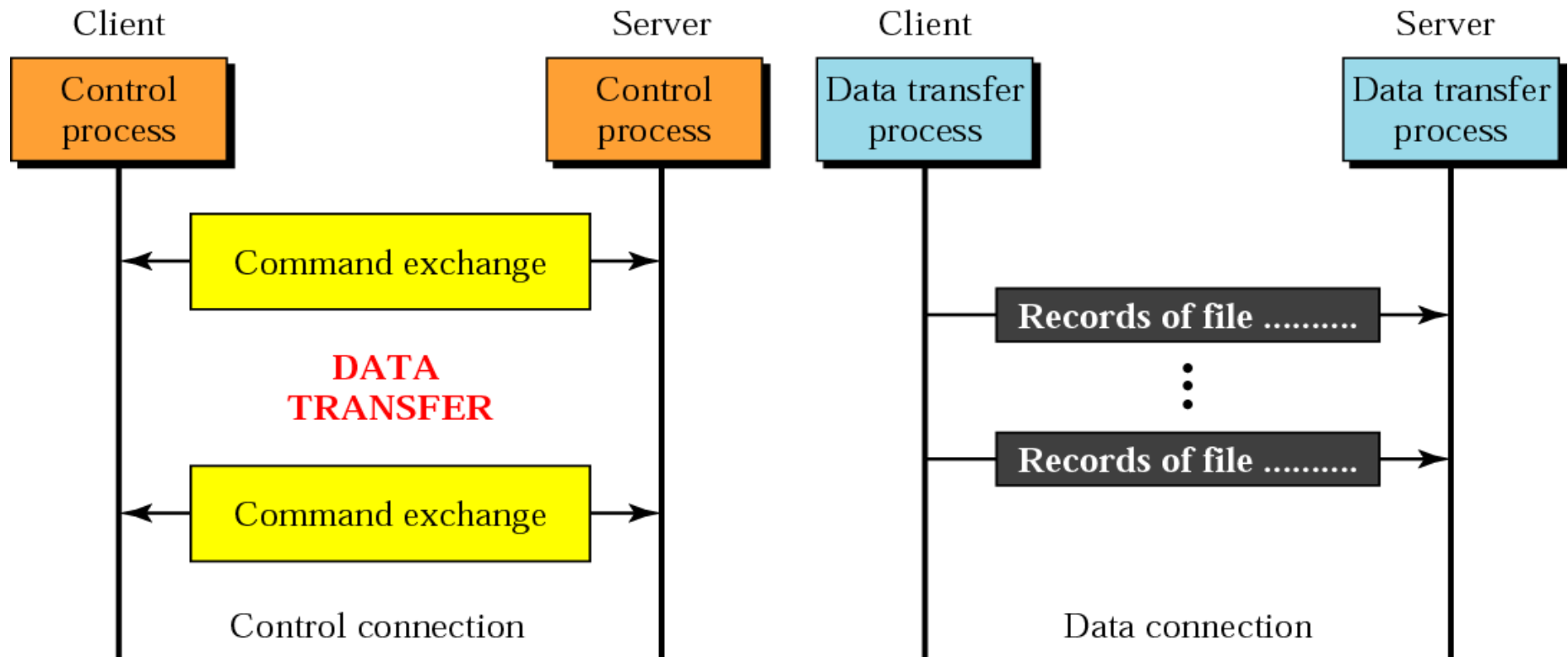


Table 26.4 *List of FTP commands in UNIX*

Commands

!, \$, account, append, ascii, bell, binary, bye, case, cd, cdup, close, cr, delete, debug, dir, discount, form, get, glob, hash, help, lcd, ls, macdef, mdelete, mdir, mget, mkdir, mls, mode, mput, nmap, ntrans, open, prompt, proxy, sendport, put, pwd, quit, quote, recv, remotehelp, rename, reset, rmdir, runique, send, status, struct, sunique, tenex, trace, type, user, verbose,?

Example 2

We show some of the user interface commands that accomplish the same task as in Example 1. The user input is shown in boldface. As shown below, some of the commands are provided automatically by the interface. The user receives a prompt and provides only the arguments.

```
$ ftp challenger.atc.fhda.edu
Connected to challenger.atc.fhda.edu
220 Server ready
Name: forouzan
Password: xxxxxxxx
ftp > ls /usr/user/report
200 OK
150 Opening ASCII mode
.....
.....
226 transfer complete
ftp > close
221 Goodbye
ftp > quit
```

Example 3

We show an example of using anonymous FTP. We connect to `internic.net`, where we assume there are some public data available.

```
$ ftp internic.net
```

```
Connected to internic.net
```

```
220 Server ready
```

```
Name: anonymous
```

```
331 Guest login OK, send "guest" as password
```

```
Password: guest
```

```
ftp > pwd
```

```
257 '/' is current directory
```

```
ftp > ls
```

```
200 OK
```

```
150 Opening ASCII mode
```

```
bin
```

```
...
```

```
ftp > close
```

```
221 Goodbye
```

```
ftp > quit
```

HTTP *and* *WWW*

27.1 HTTP

Transaction

Request Message

Response Message

Headers



HTTP uses the services of TCP on well-known port 80.

Figure 27.1 HTTP transaction

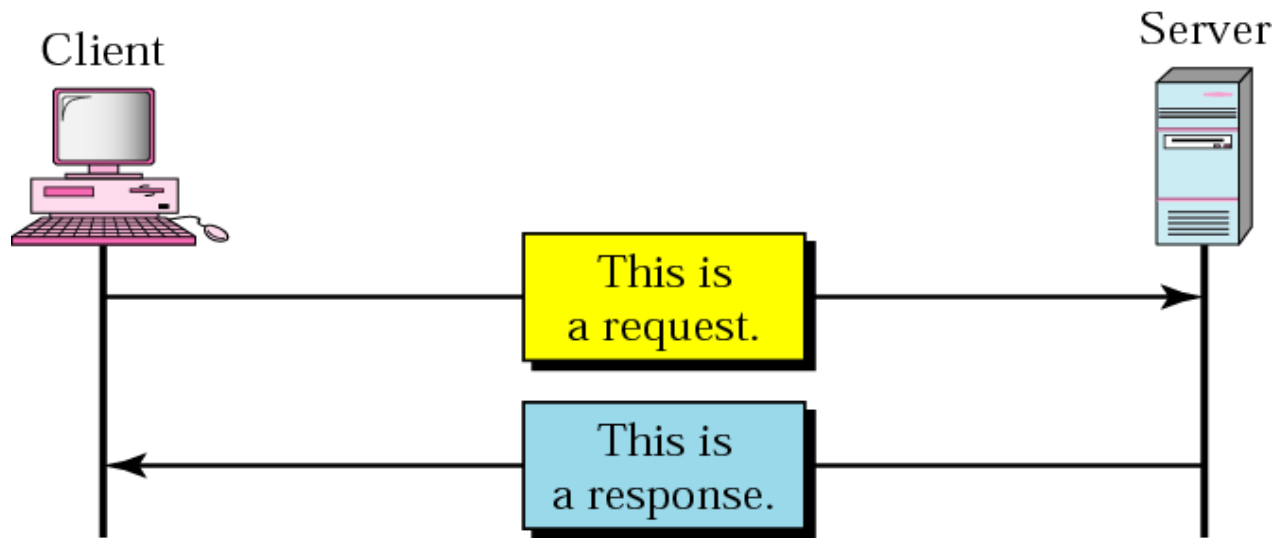


Figure 27.2 Request message

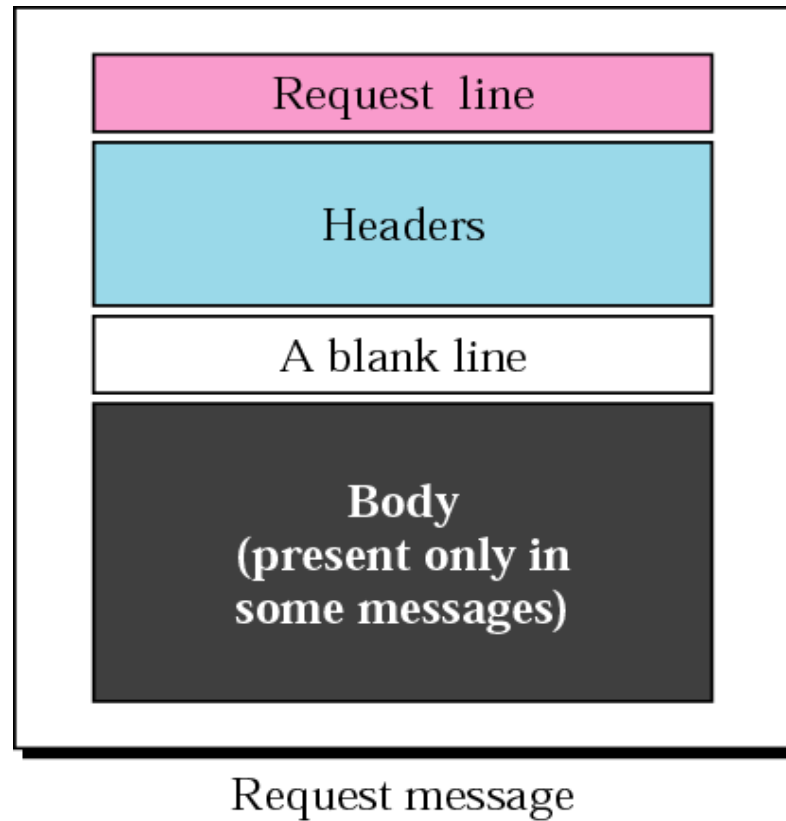


Figure 27.3 Request line



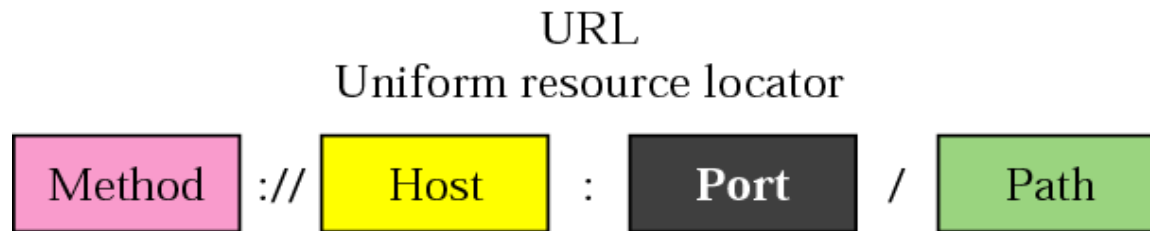


Figure 27.5 Response message

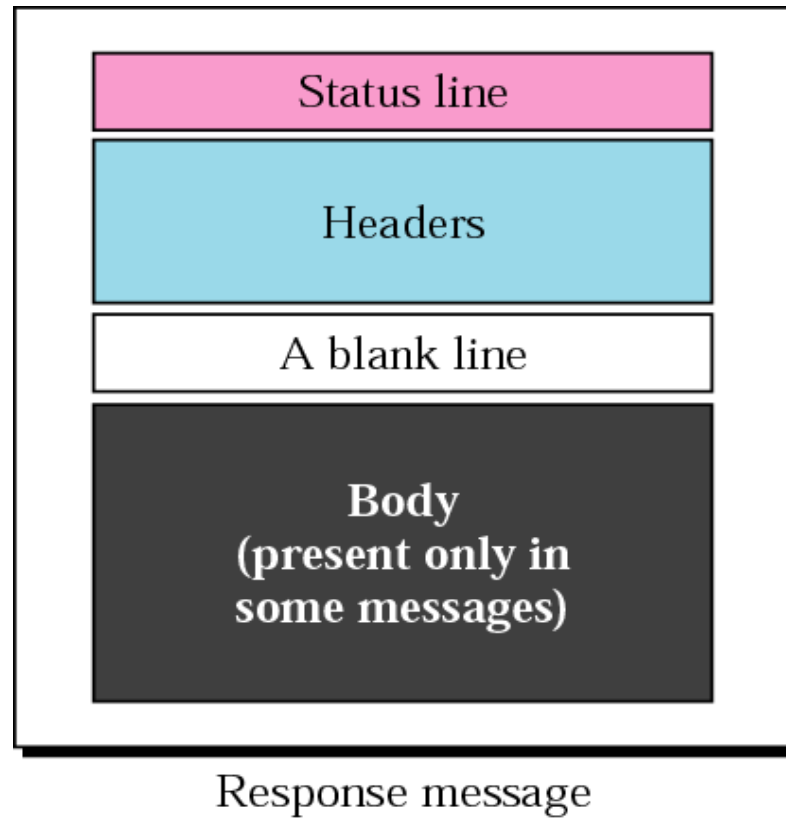


Figure 27.6 Status line

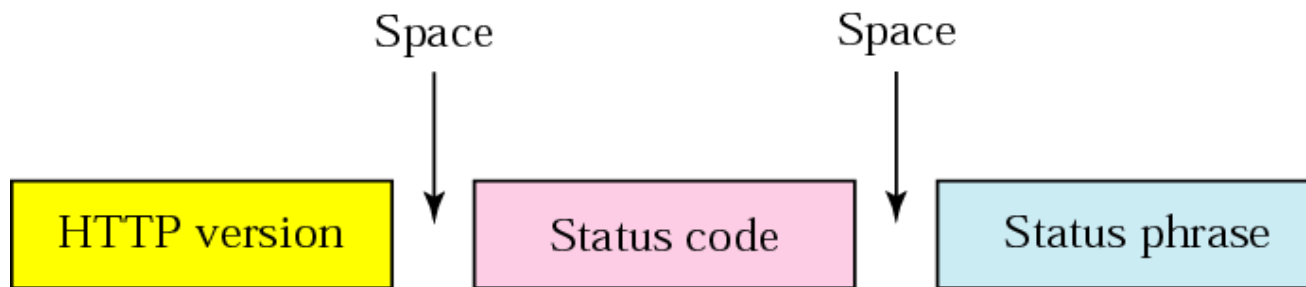


Figure 27.7 Header format

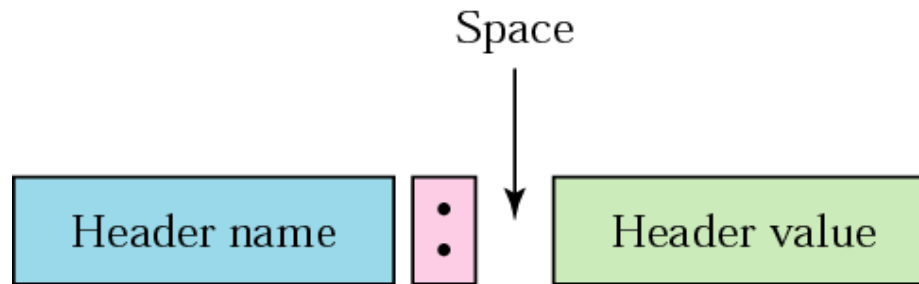
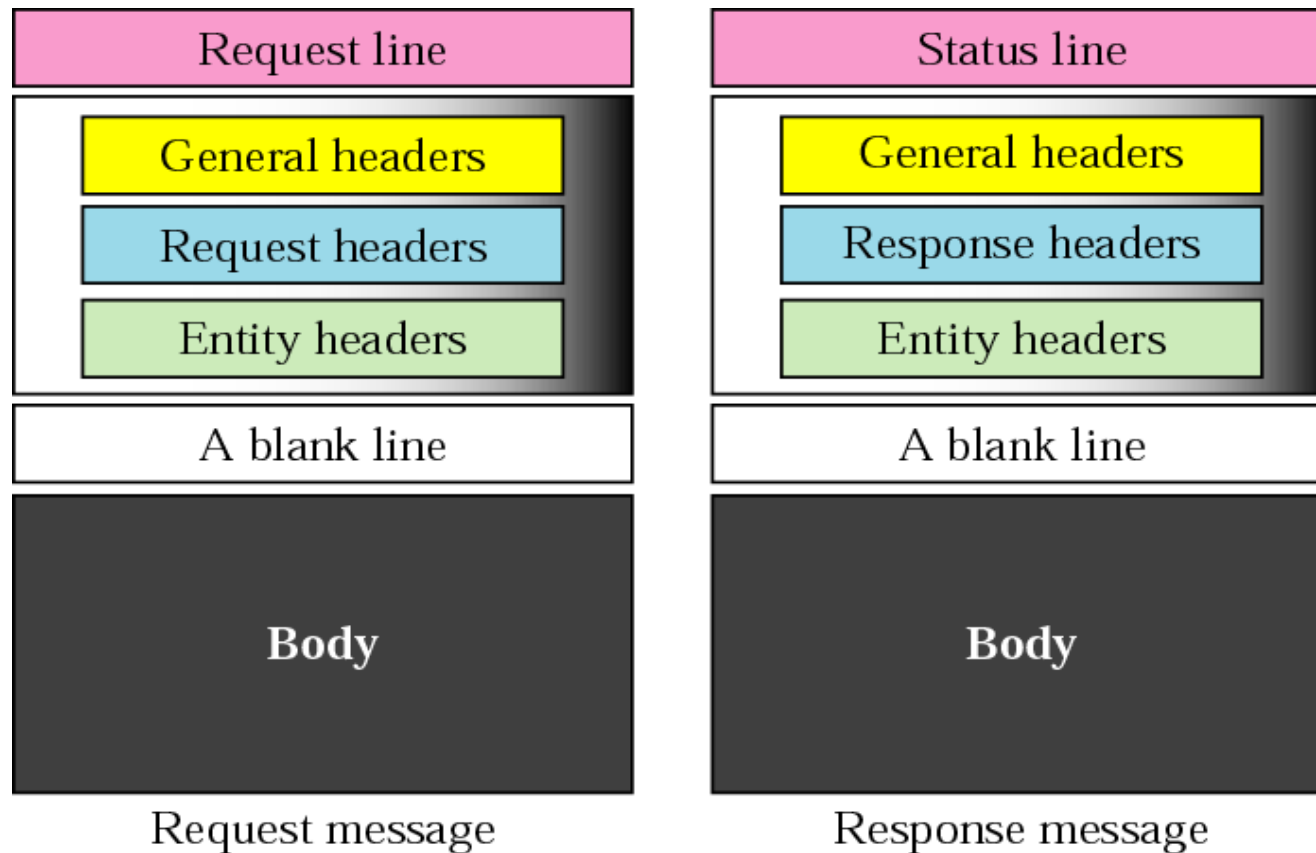


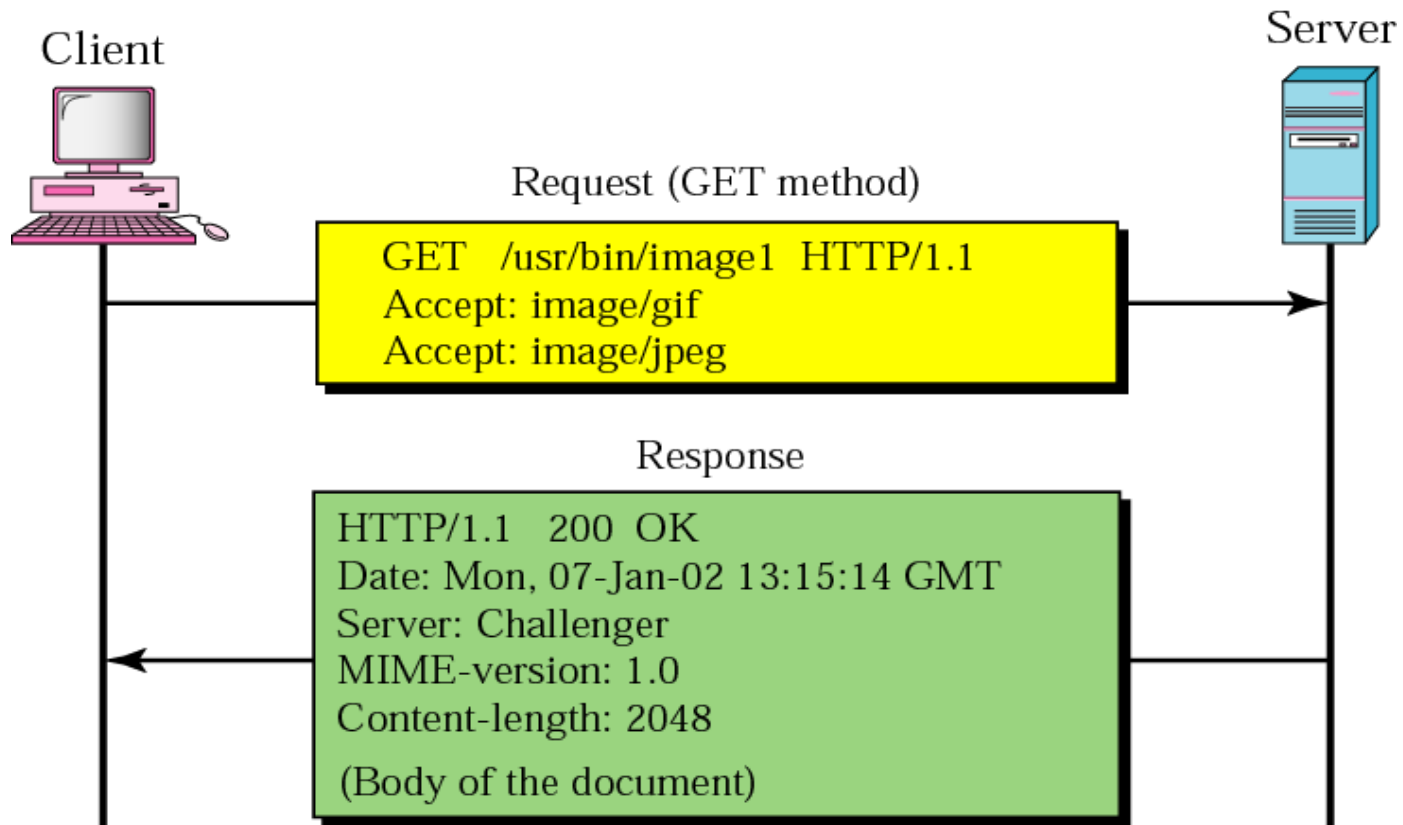
Figure 27.8 Headers



Example 1

This example retrieves a document. We use the GET method to retrieve an image with the path /usr/bin/image1. The request line shows the method (GET), the URL, and the HTTP version (1.1). The header has two lines that show that the client can accept images in GIF and JPEG format. The request does not have a body. The response message contains the status line and four lines of header. The header lines define the date, server, MIME version, and length of the document. The body of the document follows the header (see Fig. 27.9, next slide).

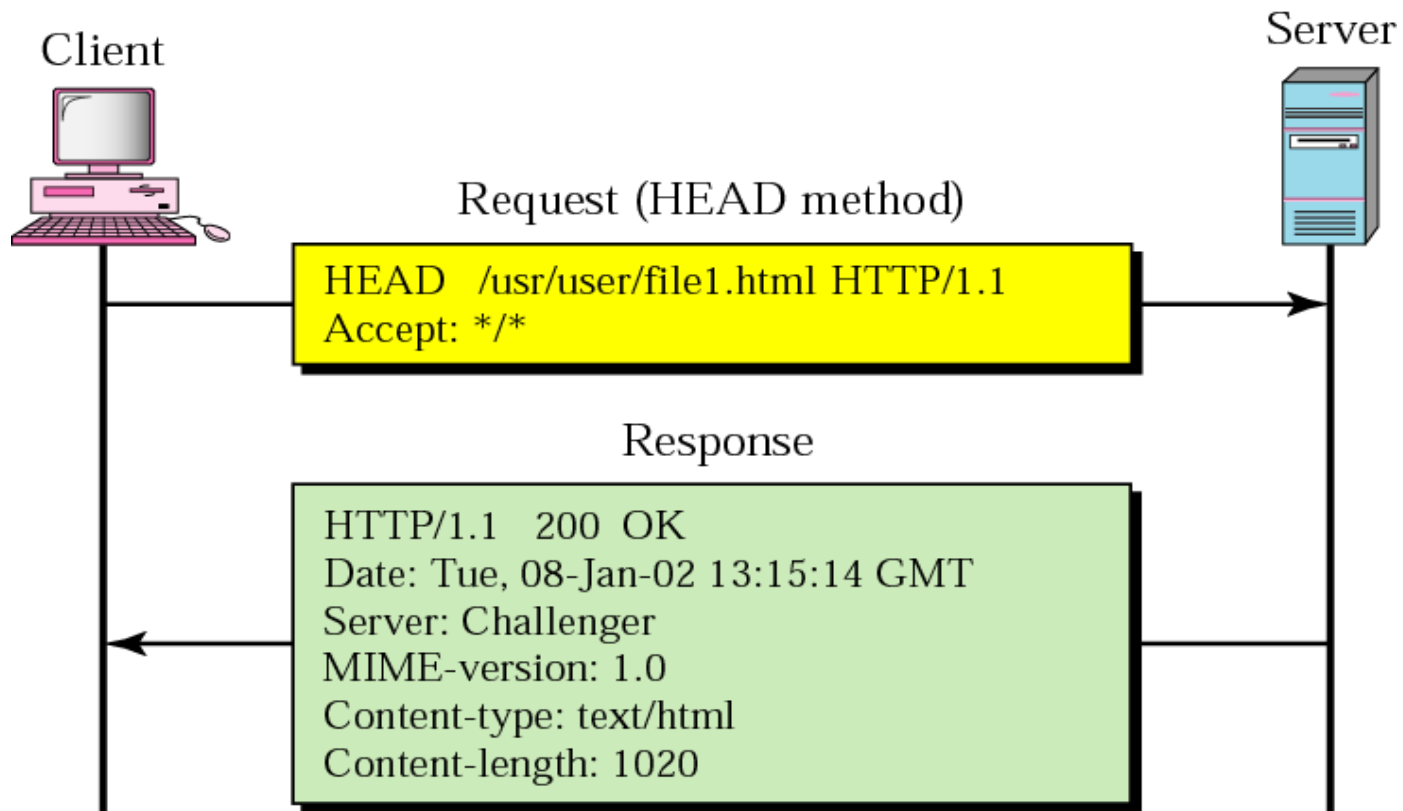
Figure 27.9 Example 1



Example 2

This example retrieves information about a document. We use the HEAD method to retrieve information about an HTML document (see the next section). The request line shows the method (HEAD), URL, and HTTP version (1.1). The header is one line showing that the client can accept the document in any format (wild card). The request does not have a body. The response message contains the status line and five lines of header. The header lines define the date, server, MIME version, type of document, and length of the document (see Fig. 27.10, next slide). Note that the response message does not contain a body.

Figure 27.10 Example 2





HTTP version 1.1 specifies a persistent connection by default.

27.2 World Wide Web

Hypertext and Hypermedia

Browser Architecture

Static Document/HTML

Dynamic Document/CGI

Active Document/Java

Figure 27.11 Distributed services

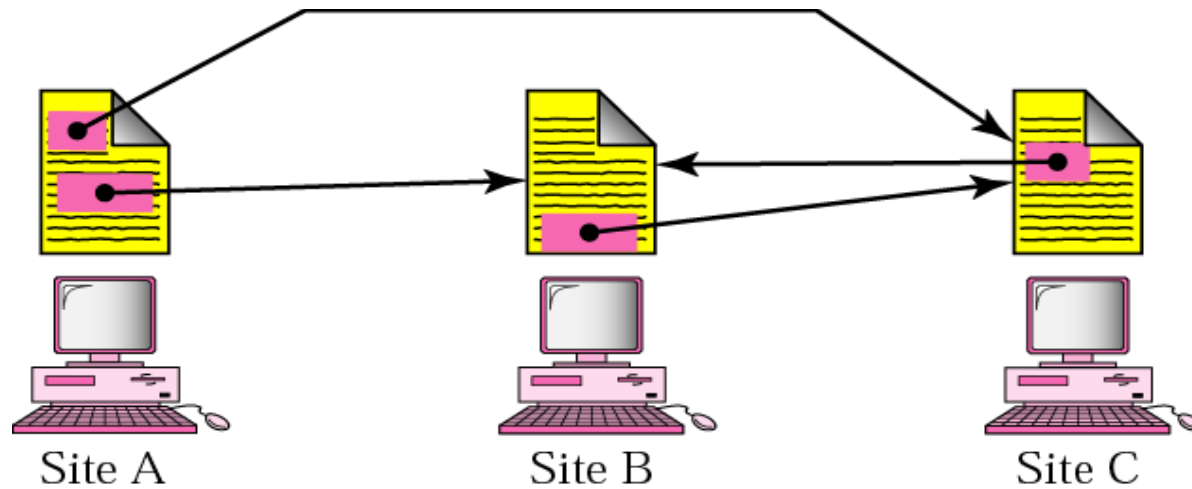


Figure 27.12 Hypertext

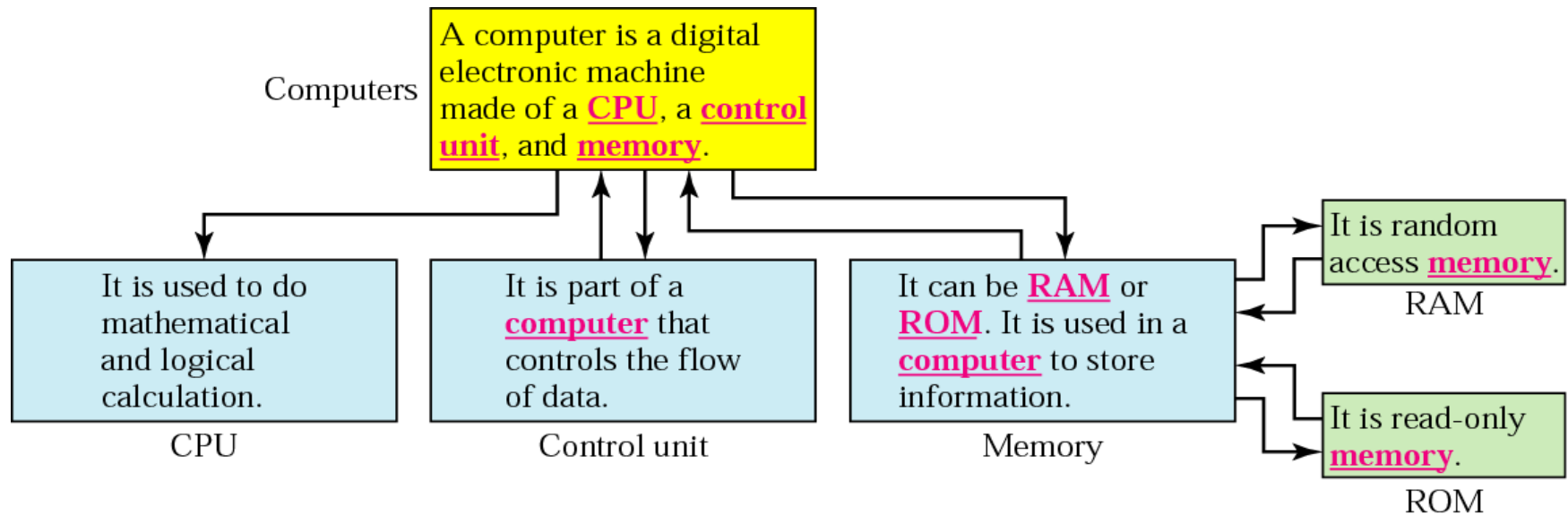


Figure 27.13 Browser architecture

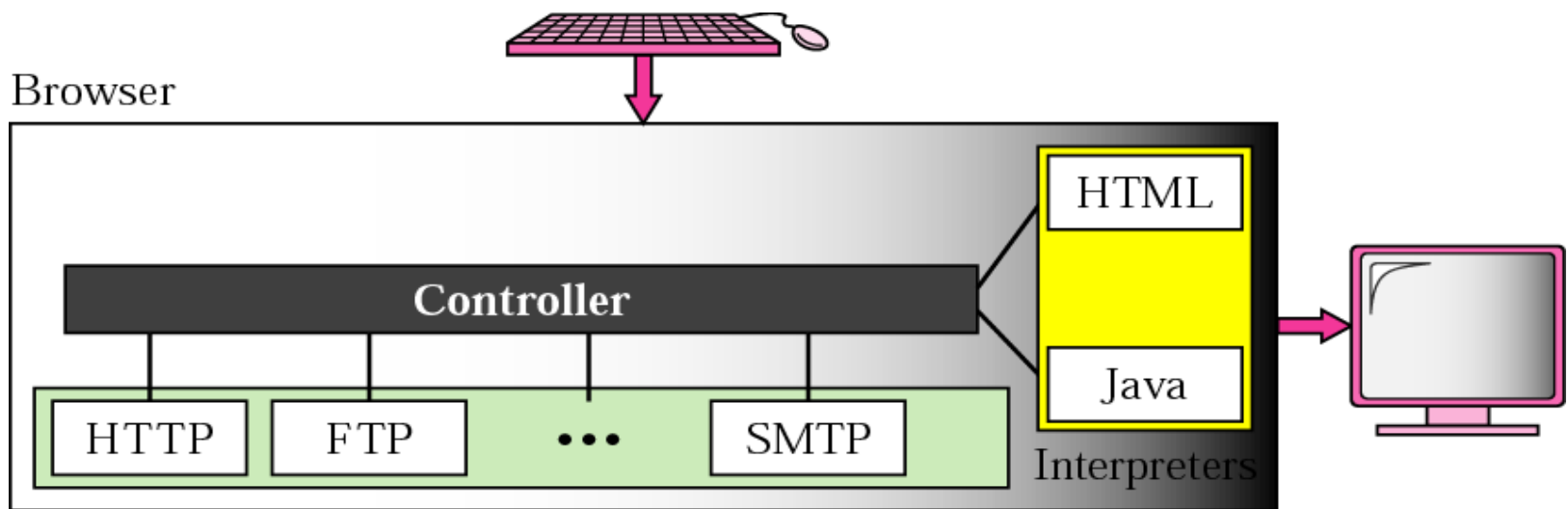


Figure 27.14 Categories of Web documents

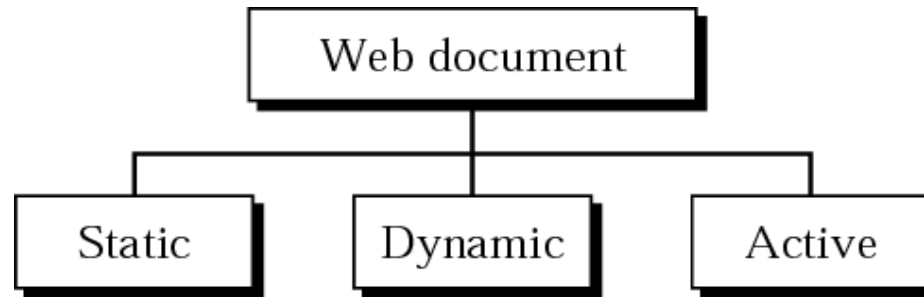


Figure 27.15 Static document

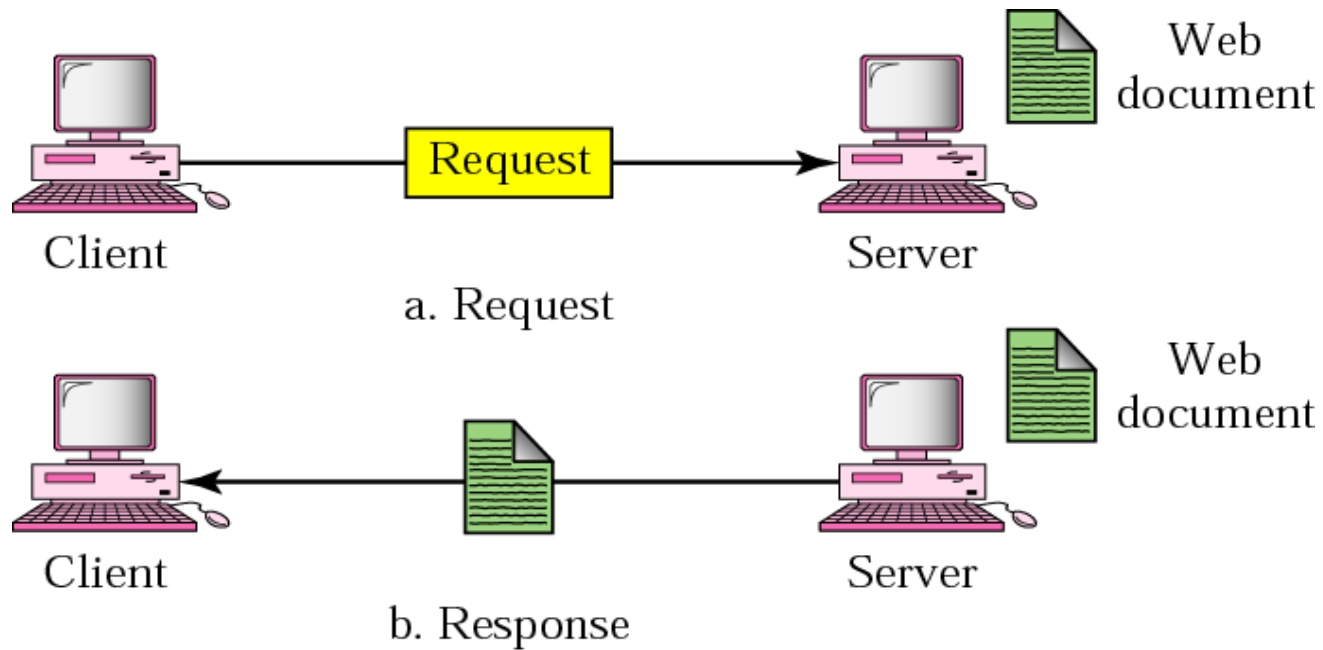


Figure 27.16 Boldface tags

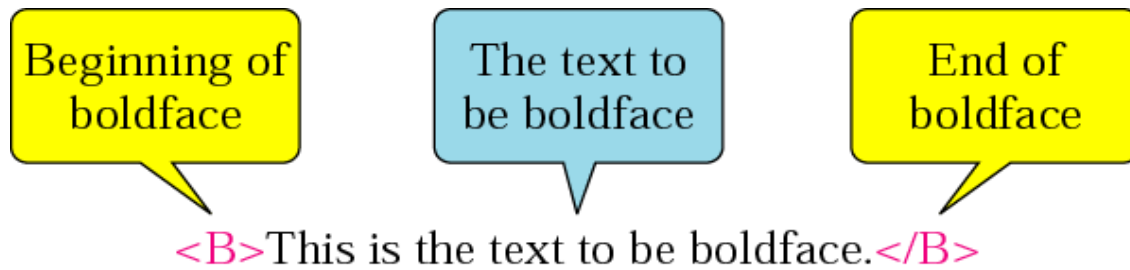


Figure 27.17 Effect of boldface tags

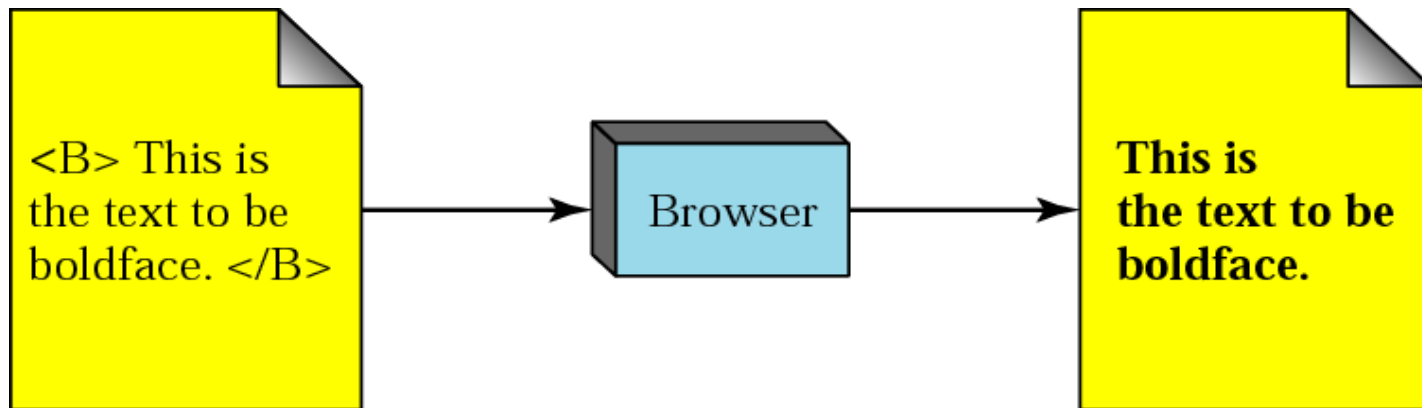


Figure 27.18 Beginning and ending tags

< TagName Attribute 5 Value Attribute 5 Value ... >

a. Beginning tag

< /TagName >

b. Ending tag

Table 27.1 Common tags

Beginning Tag	Ending Tag	Meaning
Skeletal Tags		
<HTML>	</HTML>	Defines an HTML document
<HEAD>	</HEAD>	Defines the head of the document
<BODY>	</BODY>	Defines the body of the document
Title and Header Tags		
<TITLE>	</TITLE>	Defines the title of the document
<Hn>	</Hn>	Defines the title of the document

Table 27.1 Common tags (continued)

Beginning Tag	Ending Tag	Meaning
Text Formatting Tags		
		Boldface
<I>	</I>	Italic
<U>	</U>	Underlined
_		Subscript
[	]	Superscript
Data Flow Tag		
<CENTER>	</CENTER>	Centered
 	</BR>	Line break

Table 27.1 Common tags (continued)

Beginning Tag	Ending Tag	Meaning
List Tags		
		Ordered list
		Unordered list
		An item in a list
Image Tag		
		Defines an image
Hyperlink Tag		
<A>		Defines an address (hyperlink)
Executable Contents		
<APPLET>	</APPLET>	The document is an applet

Example 3

This example shows how tags are used to let the browser format the appearance of the text.

```
<HTML>
  <HEAD>
    <TITLE> First Sample Document </TITLE>
  </HEAD>
  <BODY>
    <CENTER>
      <H1><B> ATTENTION </B></H1>
    </CENTER>
    You can get a copy of this document by:
    <UL>
      <LI> Writing to the publisher
      <LI> Ordering online
      <LI> Ordering through a bookstore
    </UL>
  </BODY>
</HTML>
```

Example 4

This example shows how tags are used to import an image and insert it into the text.

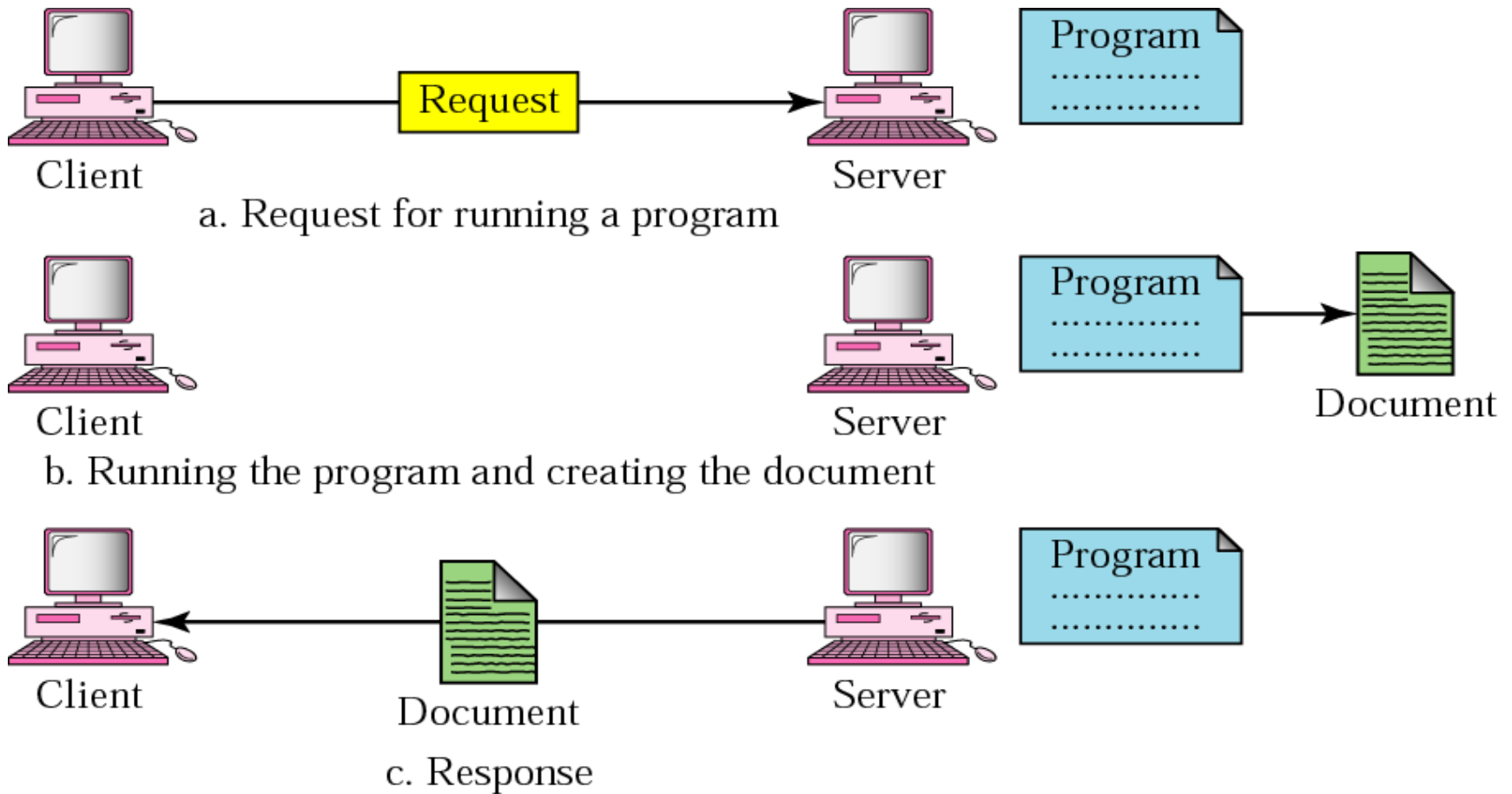
```
<HTML>
  <HEAD>
    <TITLE> Second Sample Document </TITLE>
  </HEAD>
  <BODY>
    This is the picture of a book:
    <IMG SRC="Pictures/book1.gif"  ALIGN=MIDDLE>
  </BODY>
</HTML>
```

Example 5

This example shows how tags are used to make a hyperlink to another document.

```
<HTML>
  <HEAD>
    <TITLE> Third Sample Document </TITLE>
  </HEAD>
  <BODY>
    This is a wonderful product that can save you money and time.
    To get information about the producer, click on
    <A HREF="http://www.phony.producer">
    Producer </A>
  </BODY>
</HTML>
```

Figure 27.19 Dynamic document



Example 6

Example 6 is a CGI program written in Bourne shell script. The program accesses the UNIX utility (date) that returns the date and the time. Note that the program output is in plain text.

```
#!/bin/sh
# The head of the program
echo Content_type: text/plain
echo
# The body of the program
now='date'
echo $now
exit 0
```

Example 7

Example 7 is similar to Example 6 except that program output is in HTML.

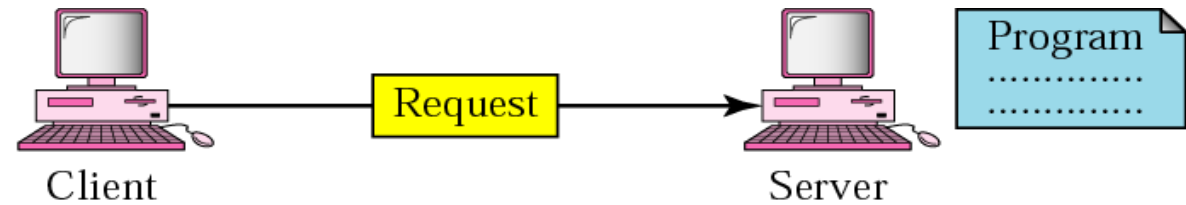
```
#!/bin/sh
# The head of the program
echo Content_type: text/html
echo
# The body of the program
echo <HTML>
echo <HEAD><TITLE> Date and Time </TITLE></HEAD>
echo <BODY>
now='date'
echo <CENTER><B> $now </B></CENTER>
echo </BODY>
echo </HTML>
exit 0
```


Example 8

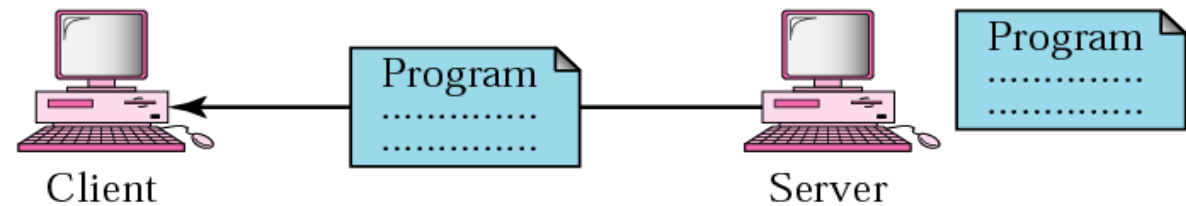
Example 8 is similar to Example 7 except that the program is written in Perl.

```
#!/bin/perl
# The head of the program
print "Content_type: text/html\n";
print "\n";
# The body of the program
print "<HTML>\n";
print "<HEAD><TITLE> Date and Time </TITLE></HEAD>\n";
print "<BODY>\n";
$now = 'date';
print "<CENTER><B> $now </B></CENTER>\n";
print "</BODY>\n";
print "</HTML>\n";
exit 0
```

Figure 27.20 Active document



a. Request for a copy of a program



b. Sending a copy of the program



c. Running the program and creating the document

Figure 27.21 Skeleton of an applet

Import libraries

public class name extends applet

{

Private data

Public methods

Private methods

}

Figure 27.22 Instantiation of the object defined by an applet

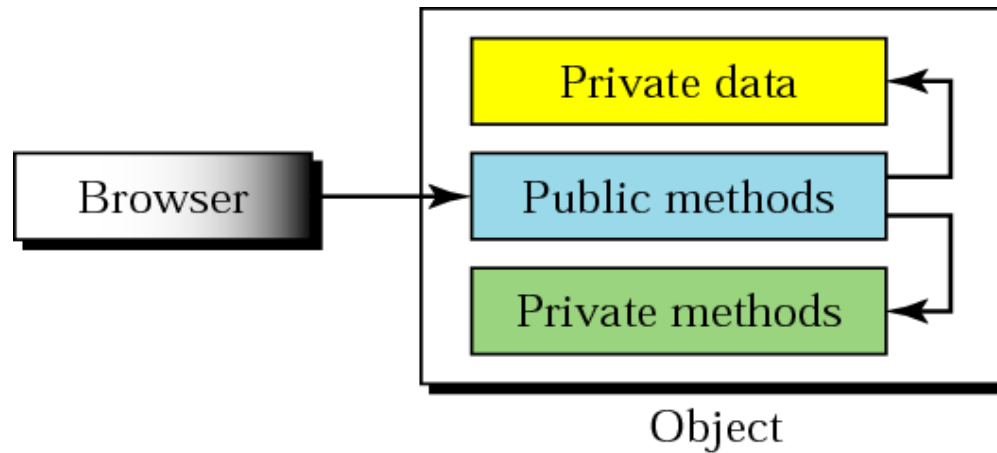


Figure 27.23 Creation and compilation

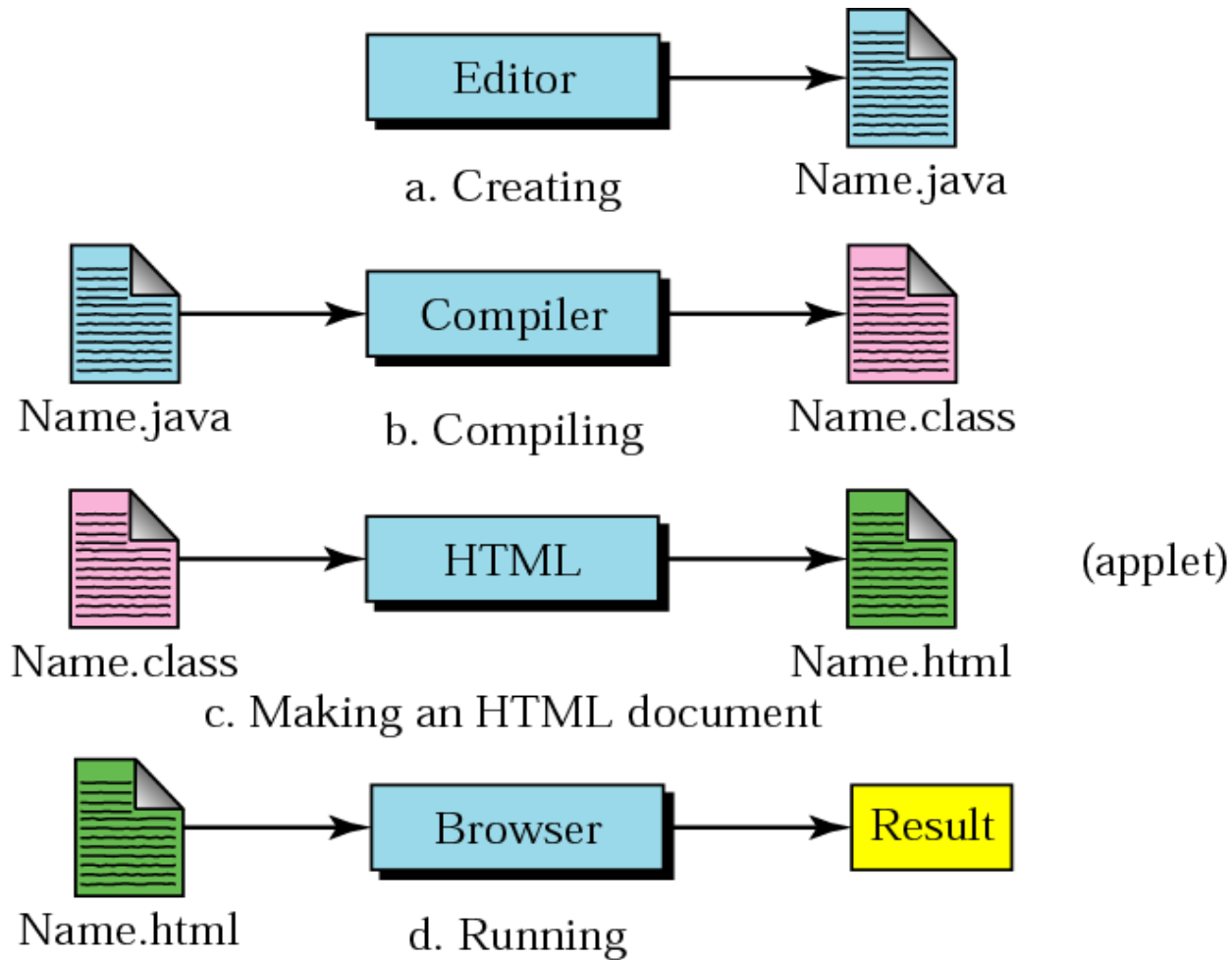


Figure 27.24 HTML document carrying an applet

```
<HTML>

  <APPLET  CODE 5 "Name.class"
  WIDTH 5 mmm
  HEIGHT 5 nnn .... >
  </APPLET >

</HTM>
```

Example 9

In this example, we first import two packages, `java.awt` and `java.applet`. They contain the declarations and definitions of classes and methods that we need. Our example uses only one publicly inherited class called `First`. We define only one public method, `paint`. The browser can access the instance of `First` through the public method `paint`. The `paint` method, however, calls another method called `drawString`, which is defined in `java.awt.*`.

```
import java.applet.*;
import java.awt.*;

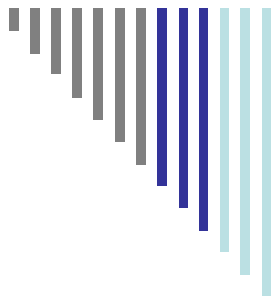
public class First extends Applet
{
    public void paint (Graphics g)
    {
        g.drawString ("Hello World", 100, 100);
    }
}
```

Example 10

In this example, we modify the program in Example 9 to draw a line. Instead of method `drawString`, we use another method called `drawLine`. This method needs four parameters: the x and y coordinates at the beginning of the line and the x and y coordinates at the end of the line. We use 0, 0 for the beginning and 80, 90 for the end.

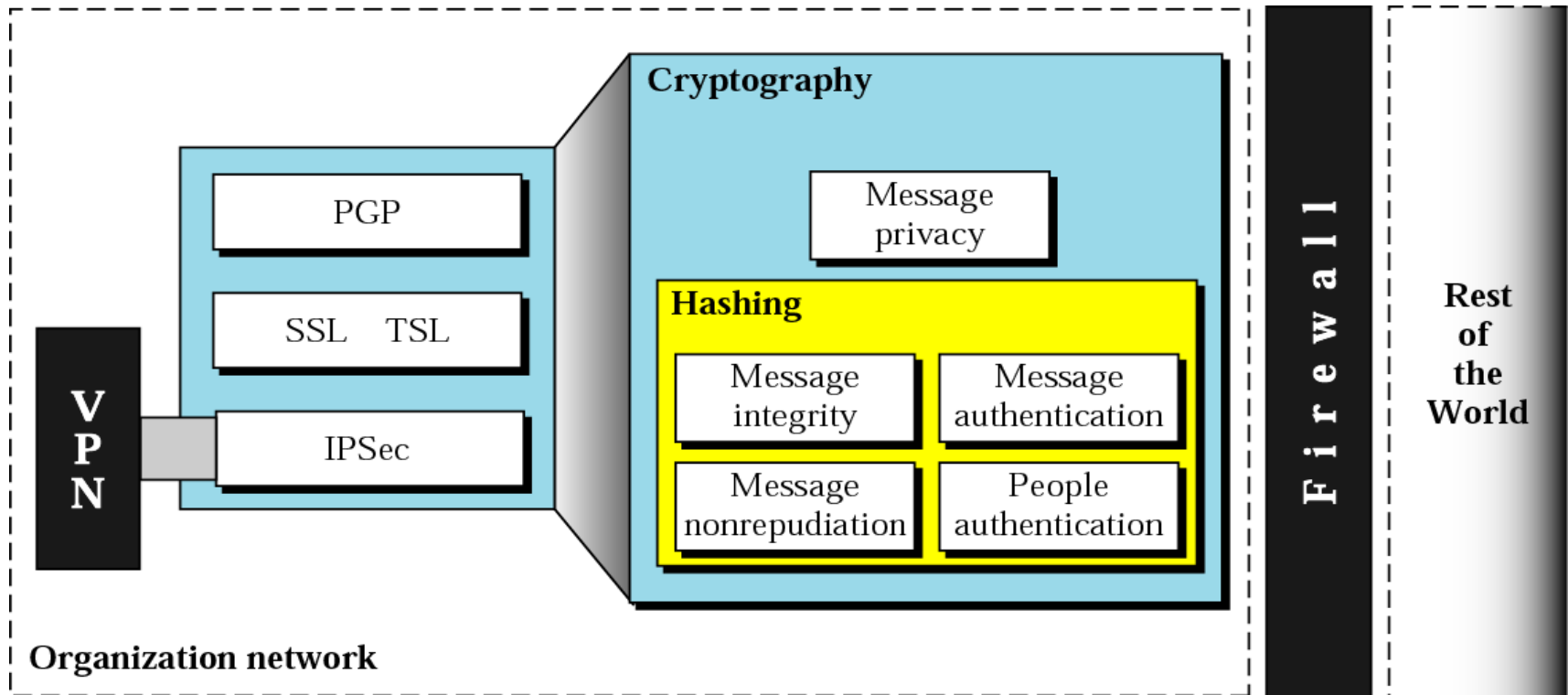
```
import java.applet.*;
import java.awt.*;

public class Second extends Applet
{
    public void paint (Graphics g)
    {
        g.drawLine (0, 0, 80, 90);
    }
}
```

Security

Security Topics



Cryptography

29.1 Introduction

Introduction to Cryptography

Figure 29.1 Cryptography components

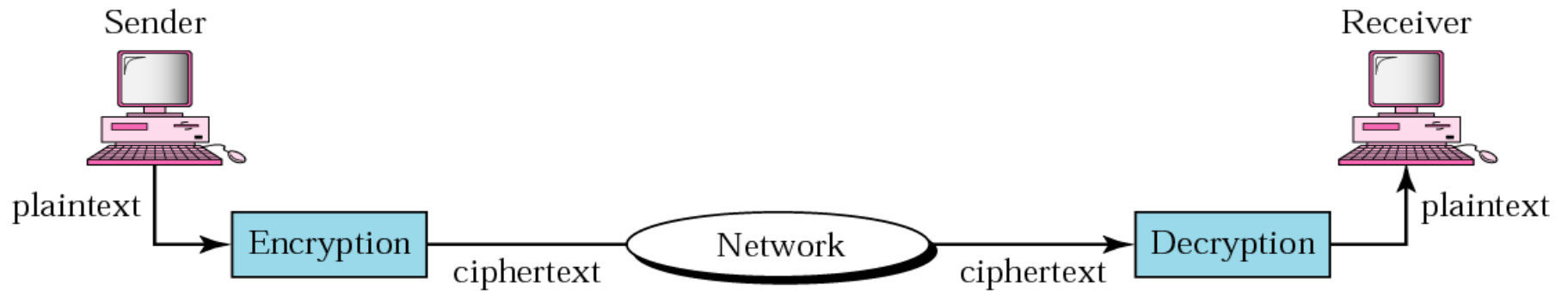
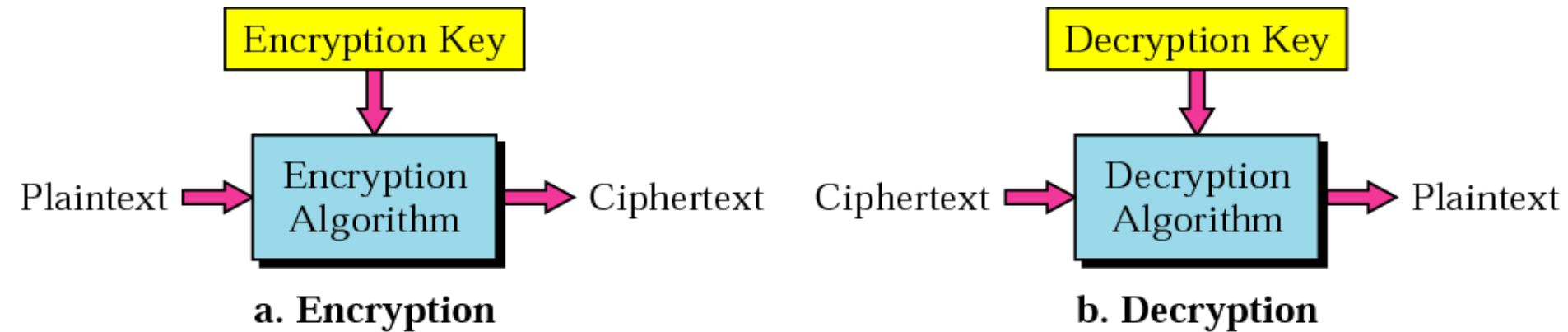


Figure 29.2 Encryption and decryption





***In cryptography,
the encryption/decryption algorithms
are public; the keys are secret.***

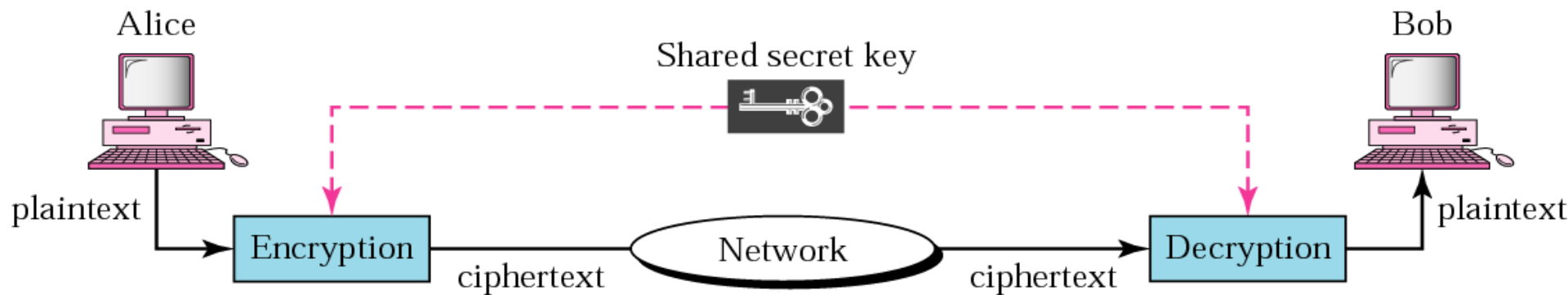
29.2 Symmetric-Key Cryptography

Traditional Cipher

Block Cipher

Operation Modes

Figure 29.3 Symmetric-key cryptography





In symmetric-key cryptography, the same key is used by the sender (for encryption) and the receiver (for decryption). The key is shared.



In symmetric-key cryptography, the same key is used in both directions.



Symmetric-key cryptography is often used for long messages.

Figure 29.4 Caesar cipher

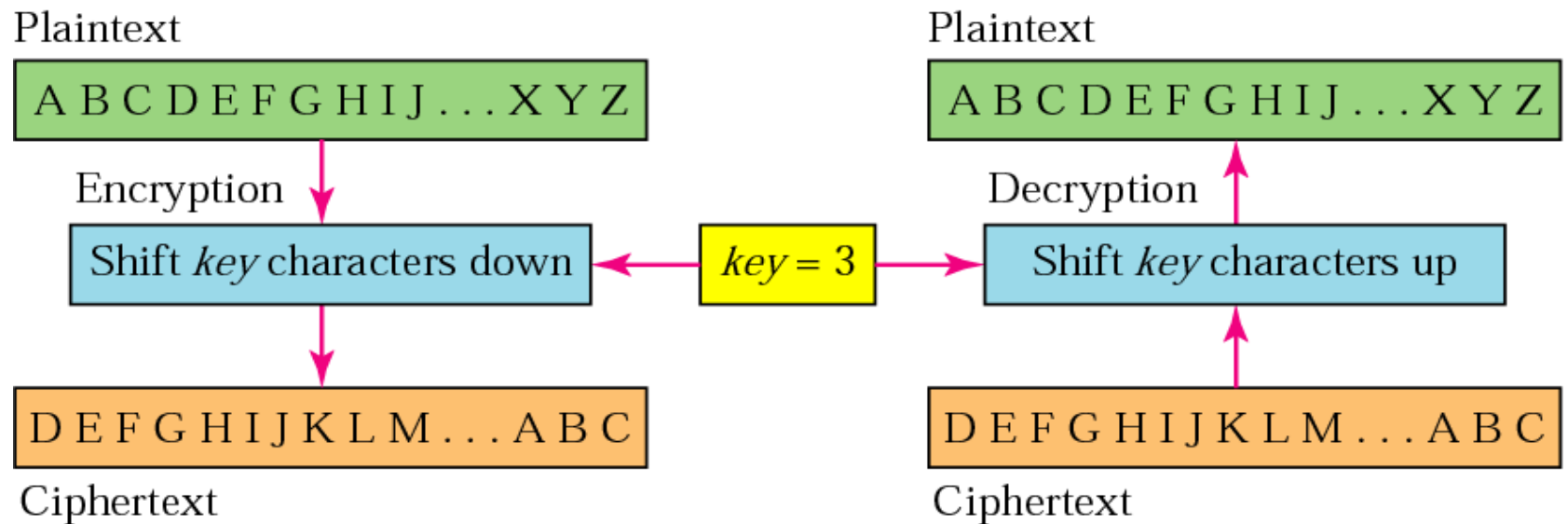


Figure 29.5 Example of monoalphabetic substitution

Encryption algorithm

Substitute top row character
with bottom row character

Decryption algorithm

Substitute bottom row character
with top row character

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
K	C	P	S	V	M	H	F	D	B	U	W	Q	N	R	Y	T	J	O	I	X	E	L	A	Z	G

Key



In monoalphabetic substitution, the relationship between a character in the plaintext to the character in the ciphertext is always one-to-one.

Figure 29.6 Vigenere cipher

Character in plaintext		A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0		W	R	K	D	O	V	C	A	S	B	Y	Q	M	L	H	I	T	U	F	E	Z	N	G	J	P	X
1		H	Q	B	G	W	E	R	K	F	C	O	A	Z	J	M	S	L	V	N	I	P	U	D	T	X	Y
2		P	I	D	Z	X	V	S	T	O	C	M	J	N	L	B	Q	R	U	W	K	H	G	E	F	A	Y
⋮		⋮																									
25		M	C	I	D	A	X	V	S	T	O	N	L	K	U	R	E	W	Z	H	F	P	G	Y	J	B	Q

Character in Ciphertext

Key = (Position of character in the text) mod 26



In polyalphabetic substitution, the relationship between a character in the plaintext and a character in the ciphertext is one-to-many.

Figure 29.7 Transpositional cipher

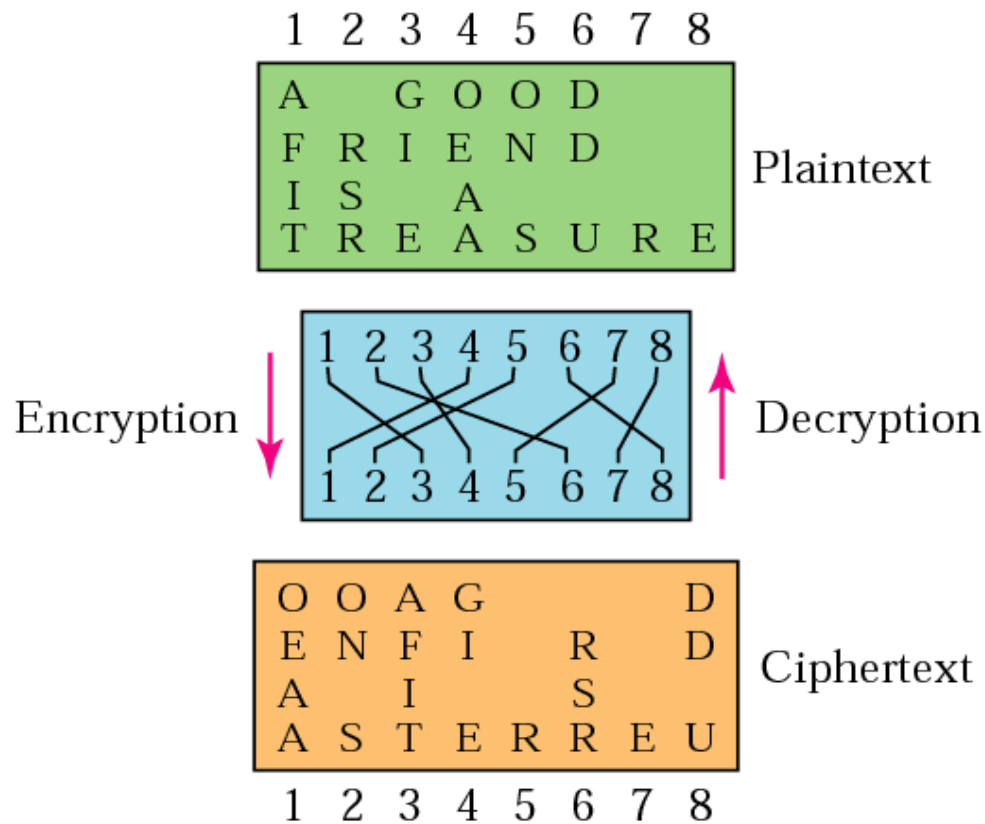


Figure 29.8 Block cipher

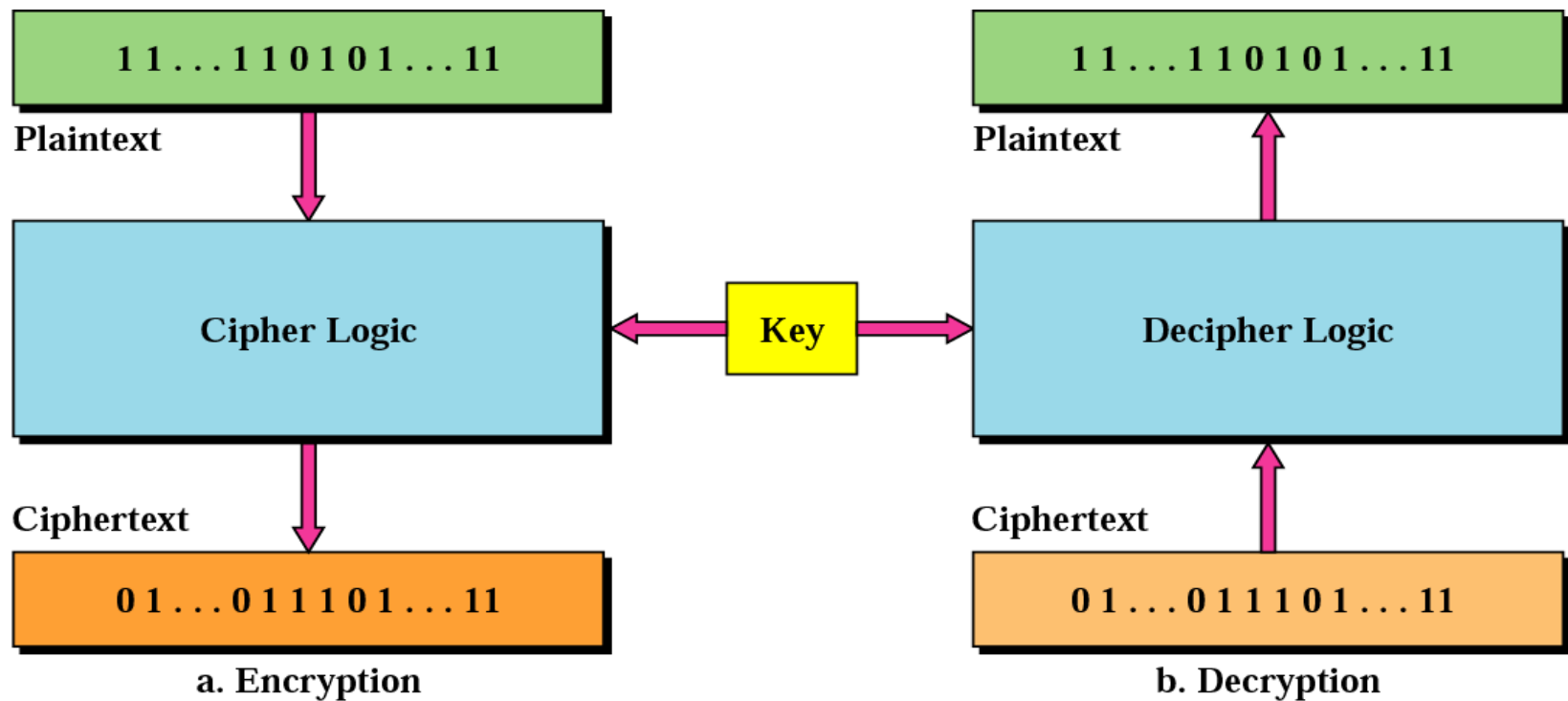


Figure 29.9 P-box

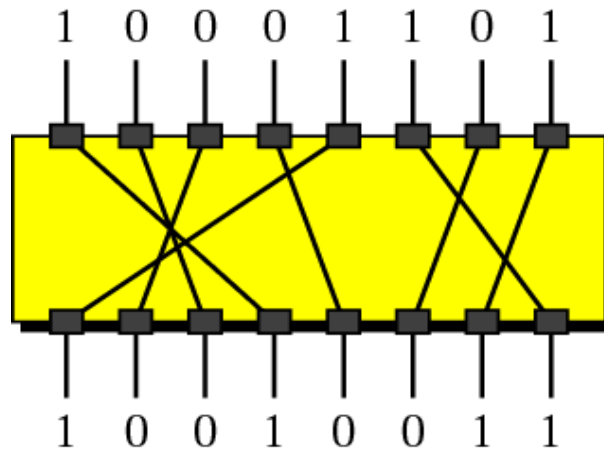


Figure 29.10 S-box

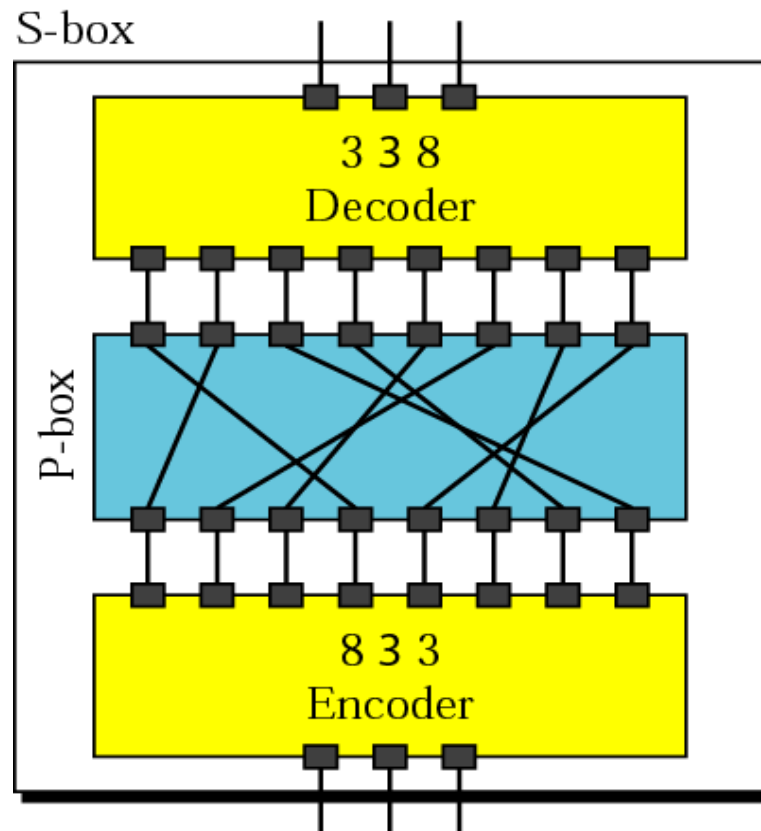


Figure 29.11 Product block

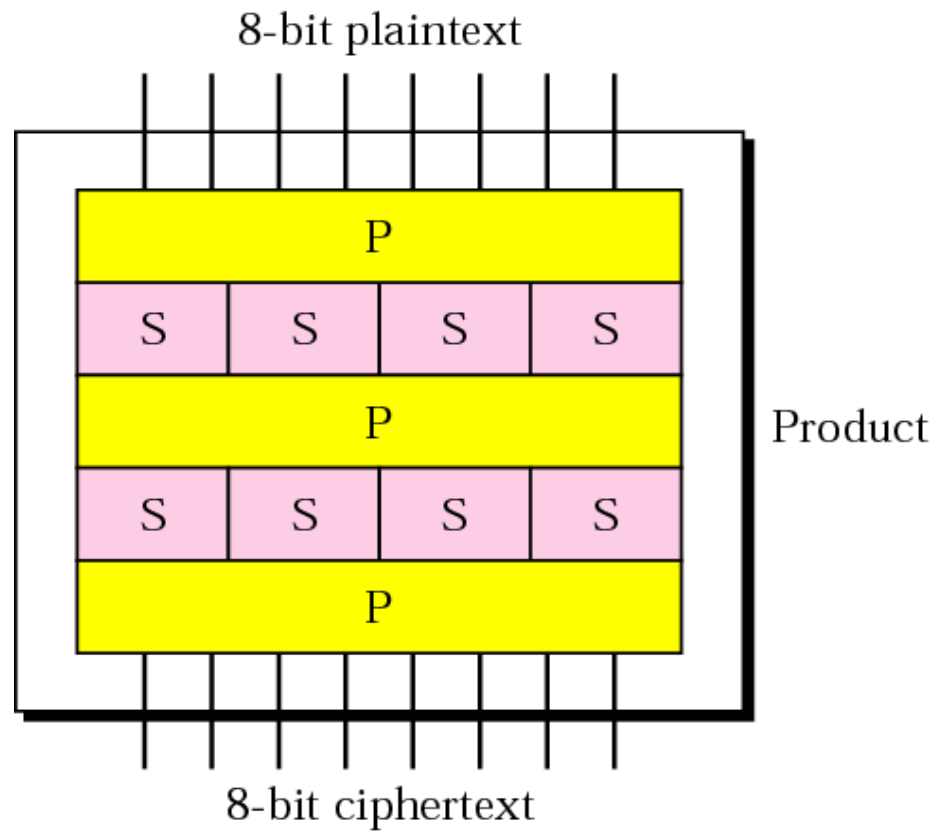


Figure 29.12 DES

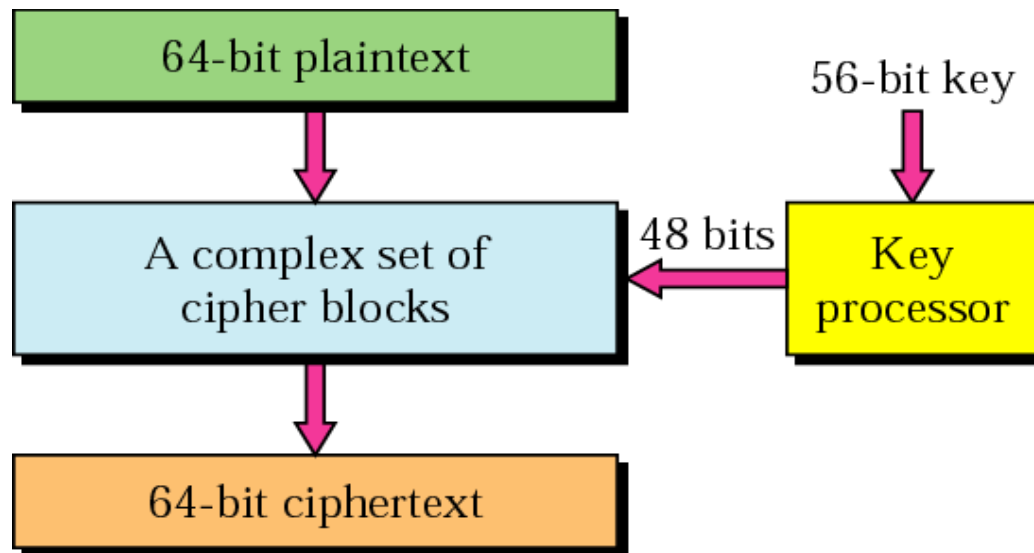


Figure 29.13 General scheme of DES

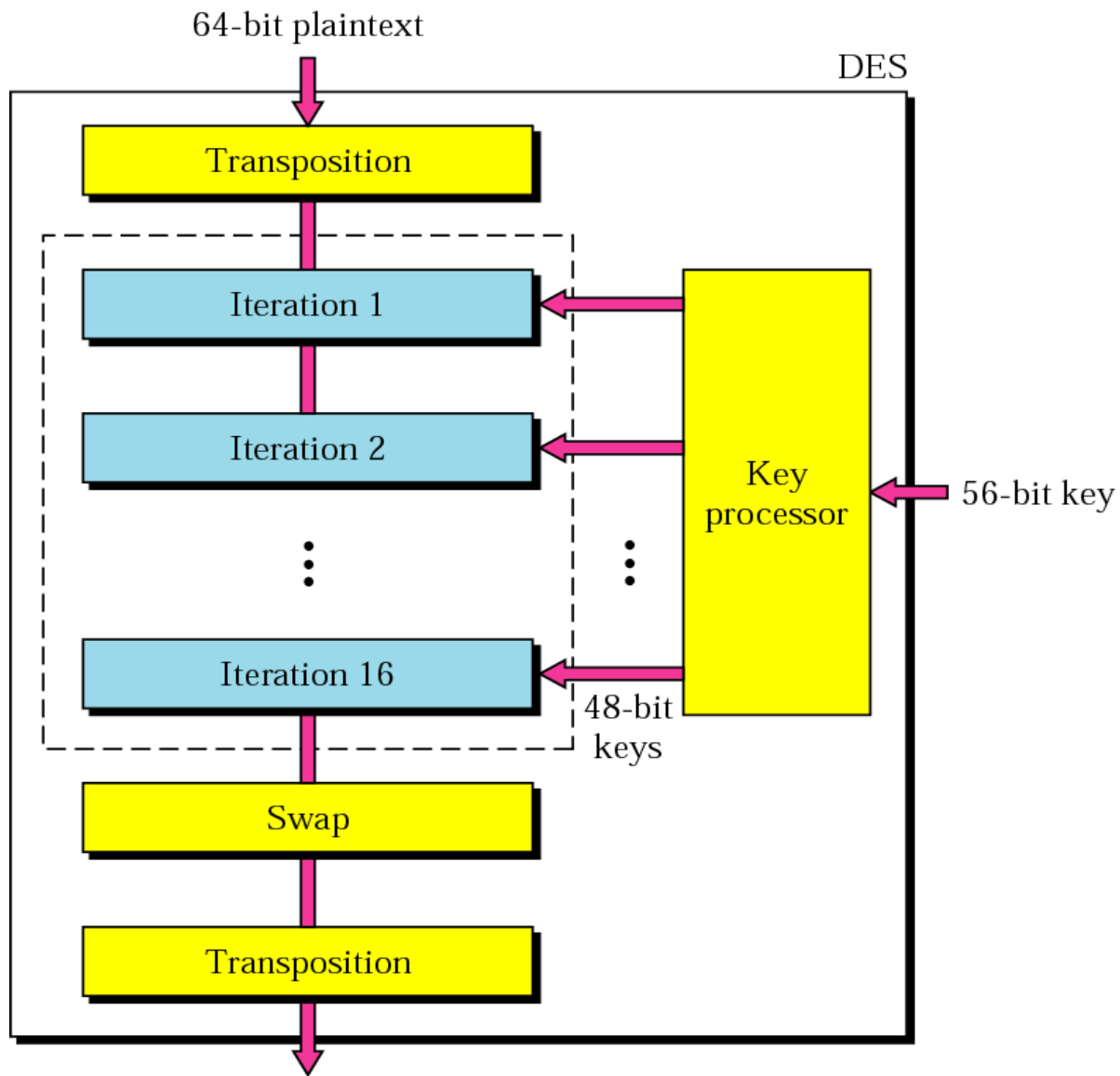


Figure 29.14 Iteration block

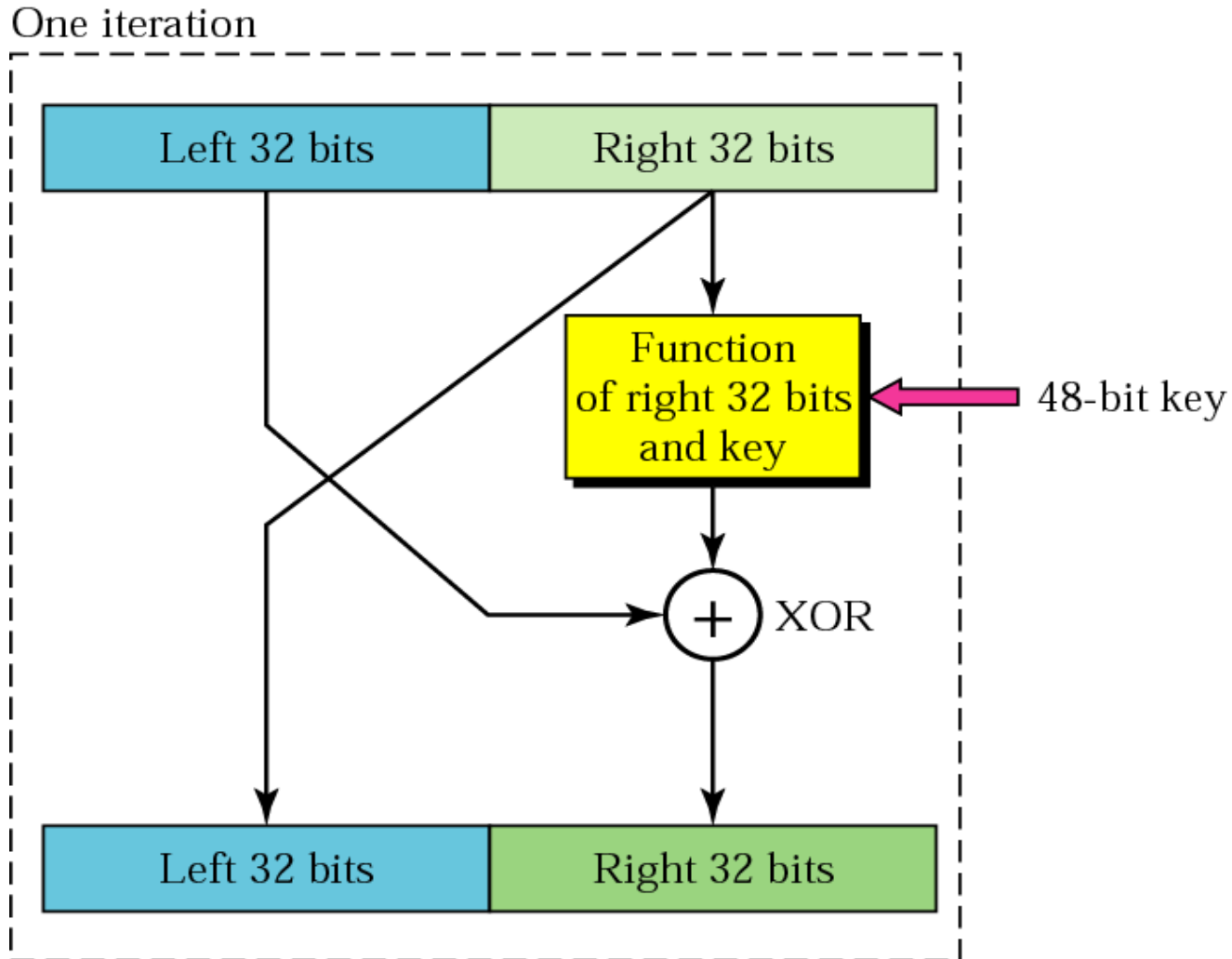
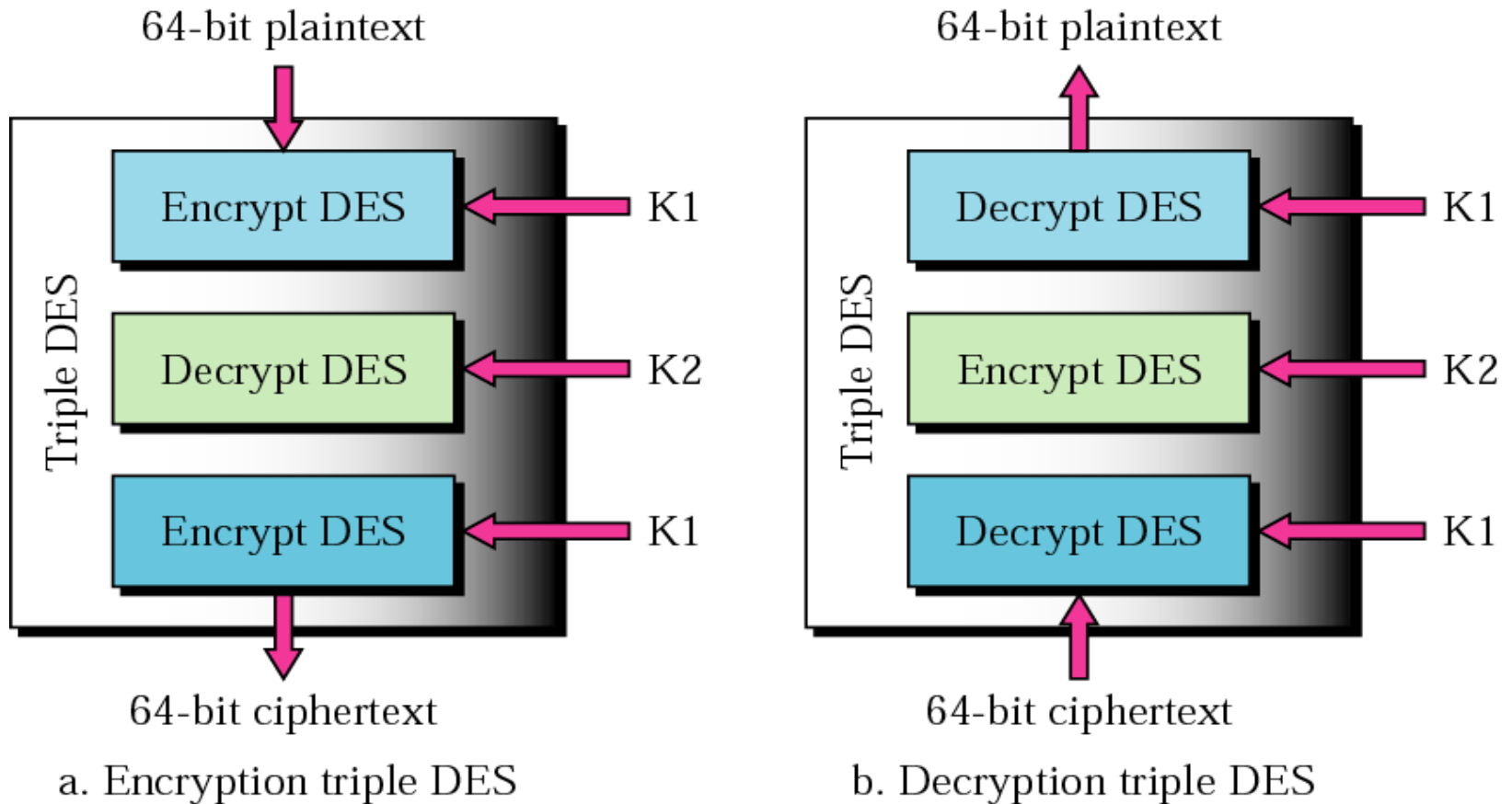


Figure 29.15 Triple DES





The DES cipher uses the same concept as the Caesar cipher, but the encryption/decryption algorithm is much more complex due to the sixteen 48-bit keys derived from a 56-bit key.

Figure 29.16 ECB mode

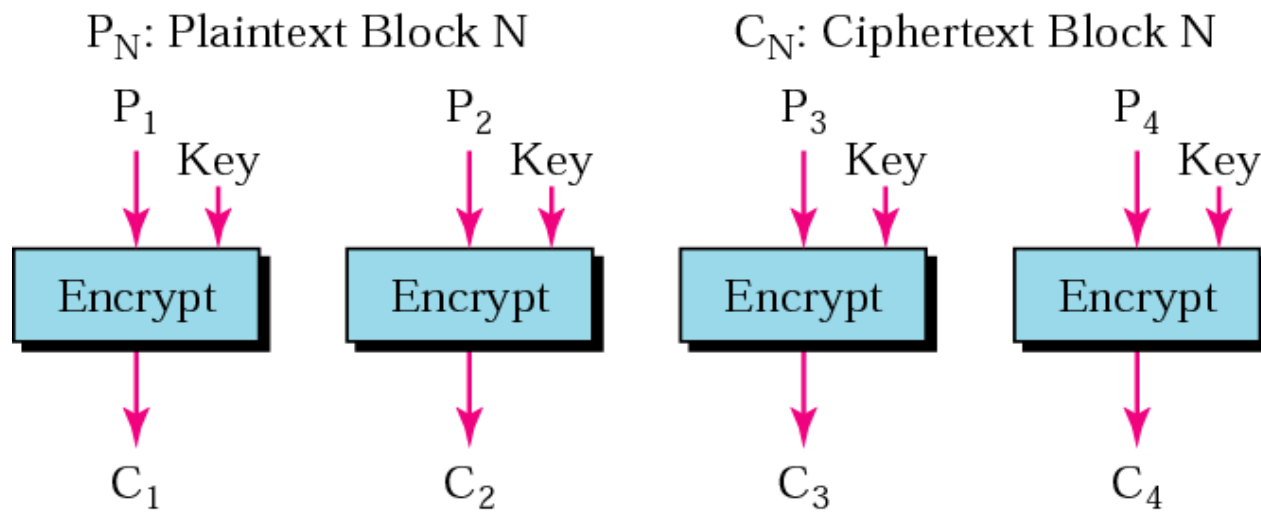
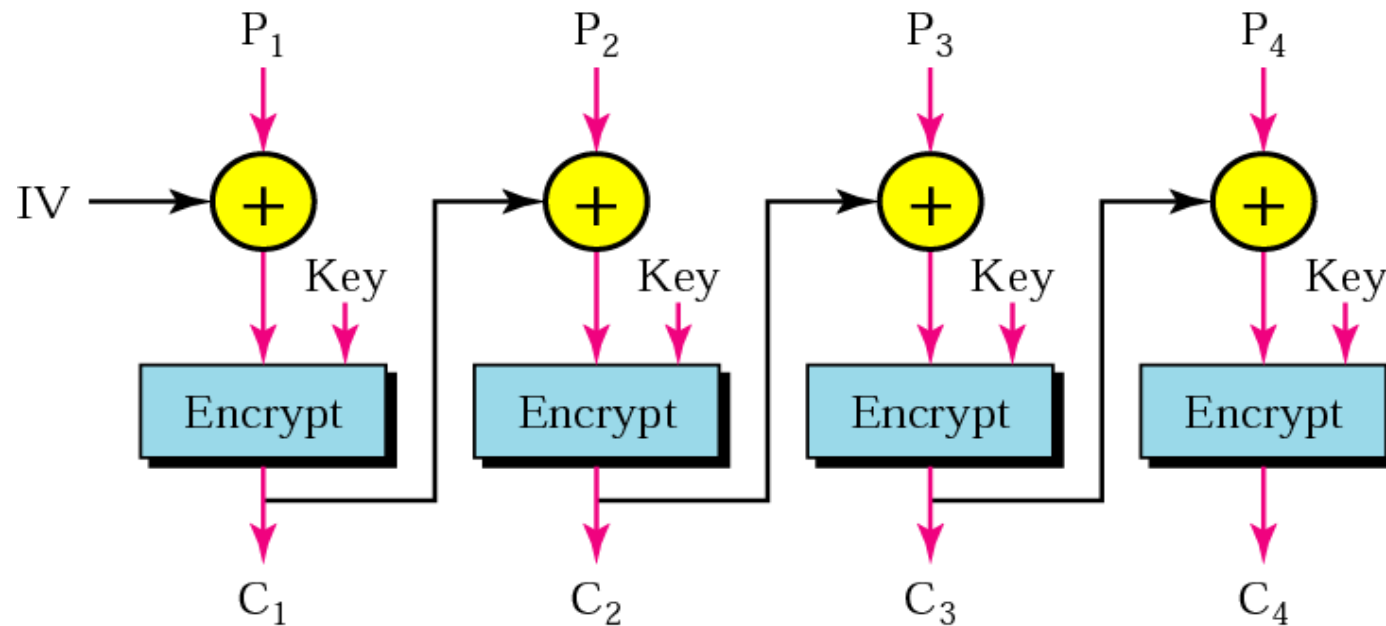


Figure 29.17 CBC mode

IV: Initialization Vector P_N : Plaintext Block N C_N : Ciphertext Block N



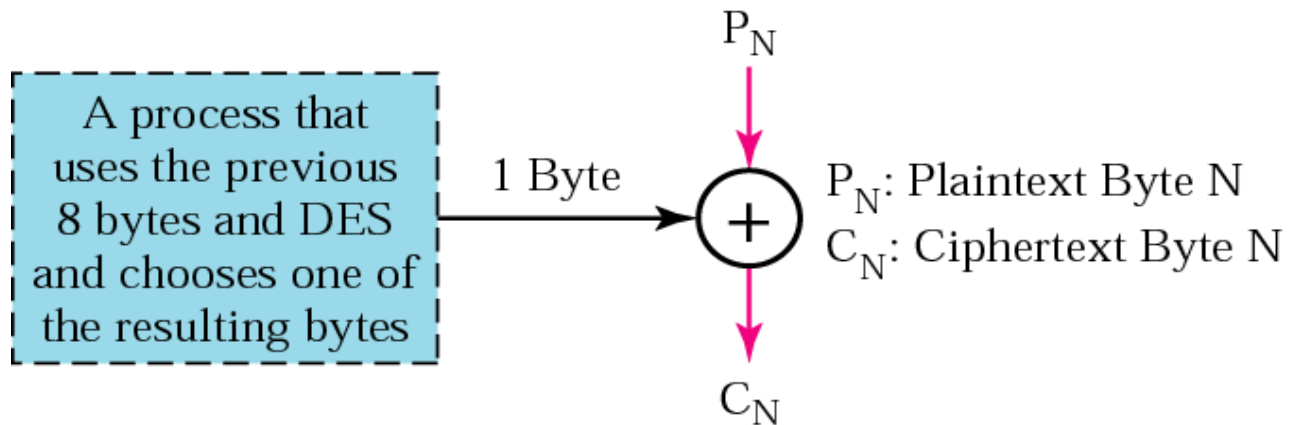
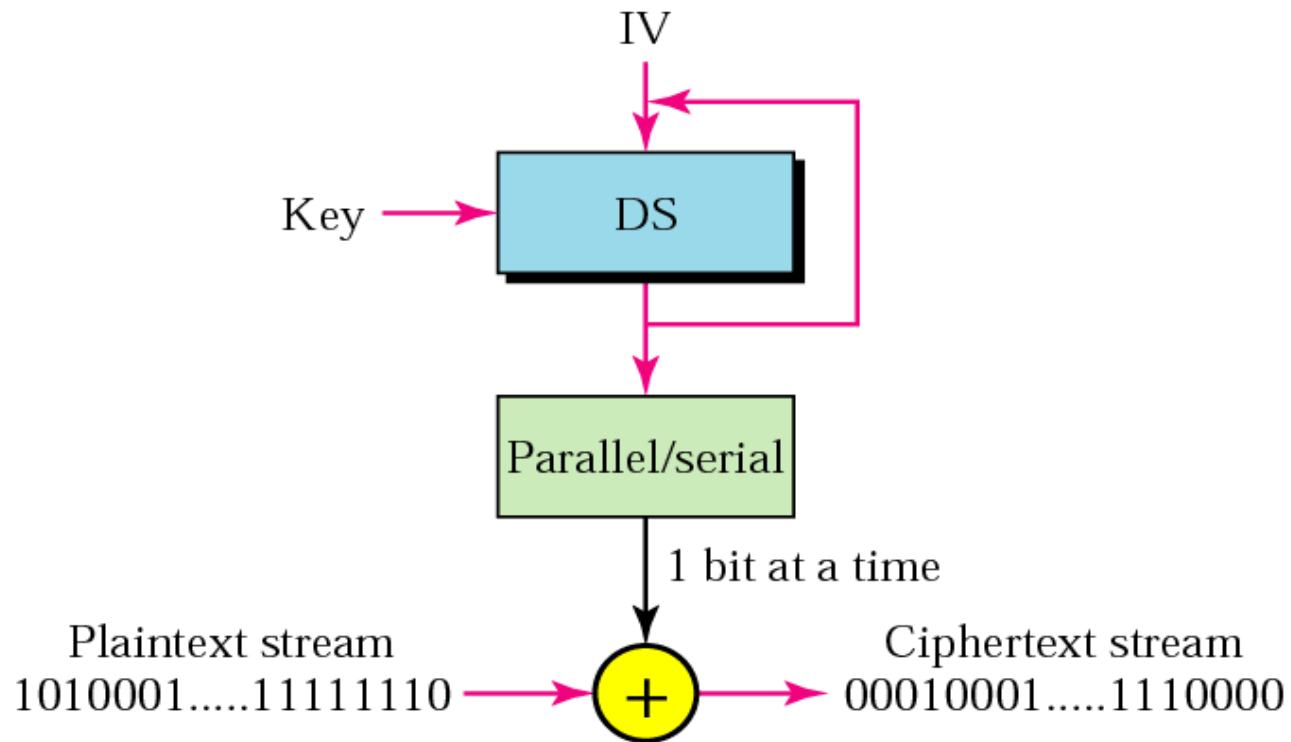


Figure 29.19 CSM

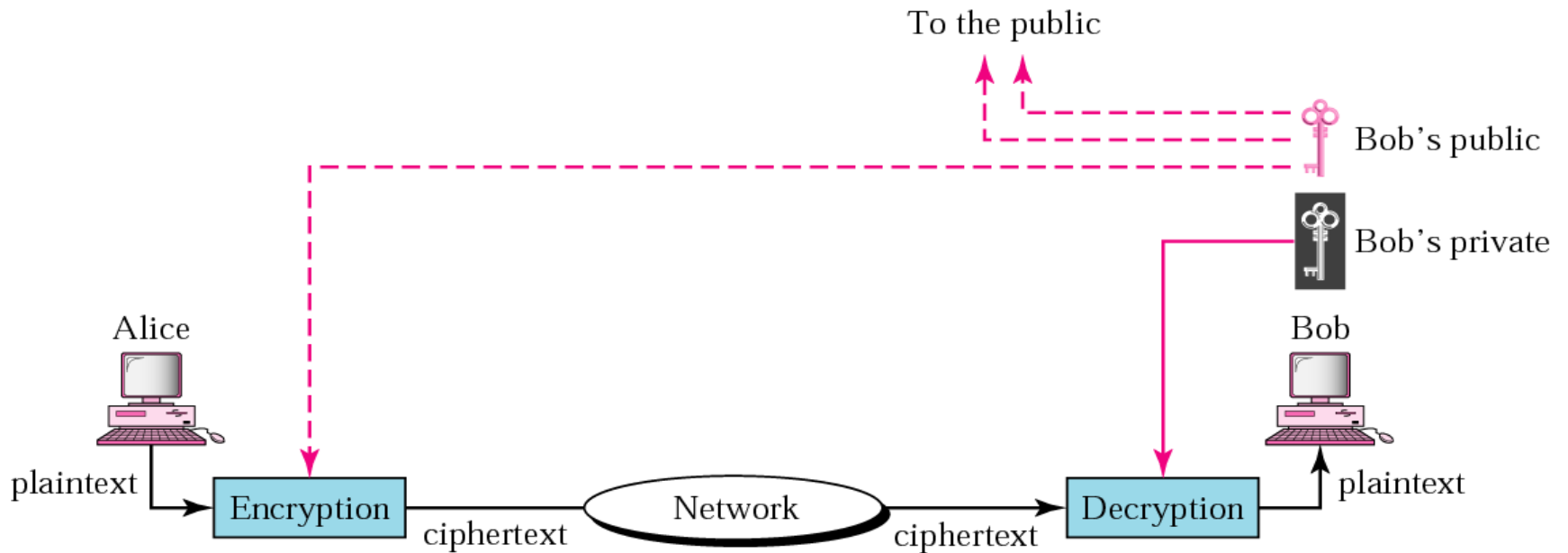


29.3 Public-Key Cryptography

RSA

Choosing Public and Private Keys

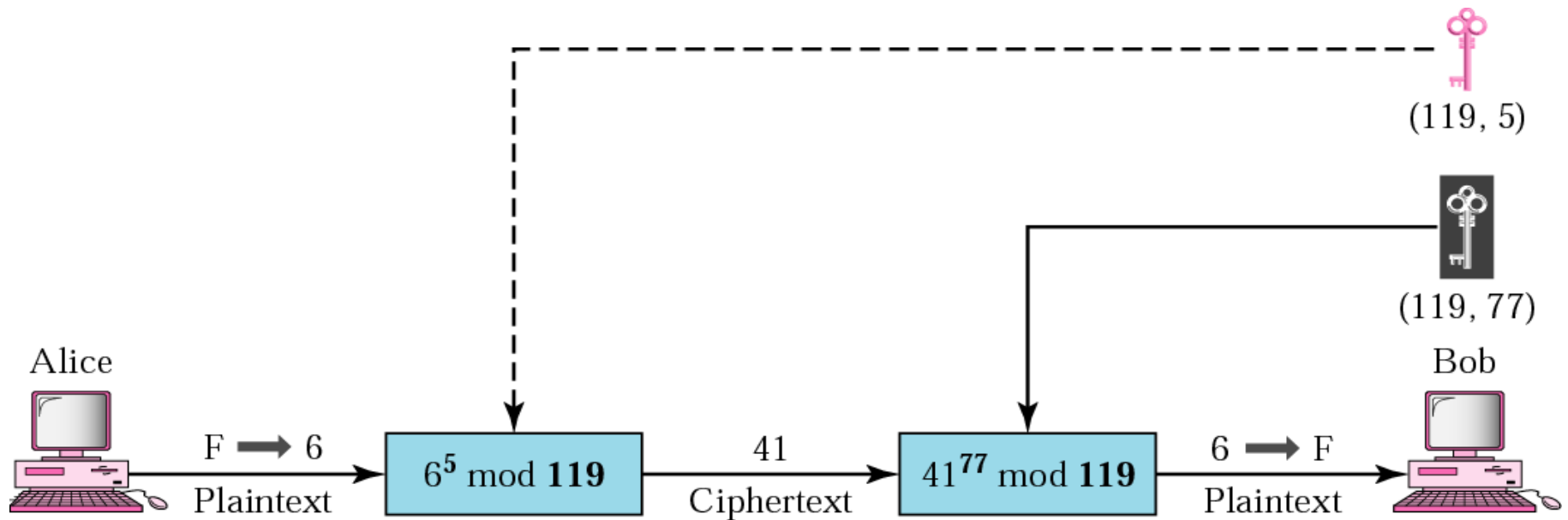
Figure 29.20 Public-key cryptography

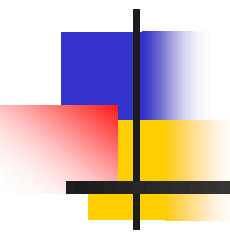




Public-key algorithms are more efficient for short messages.

Figure 29.21 RSA





END

(Effort Never Dies)